podlodka

AI Crew
12 - 16 октября

Мультиагентные системы

Максим Качурин

Co-Founder @Reforma

CTO @flexbe

Доклад

Как архитектура кодинг-агента определяет его стоимость

RWB Selectel

АВТОР

Максим Качурин

Сооснователь reforma

СТО Конструктор сайтов flexbe

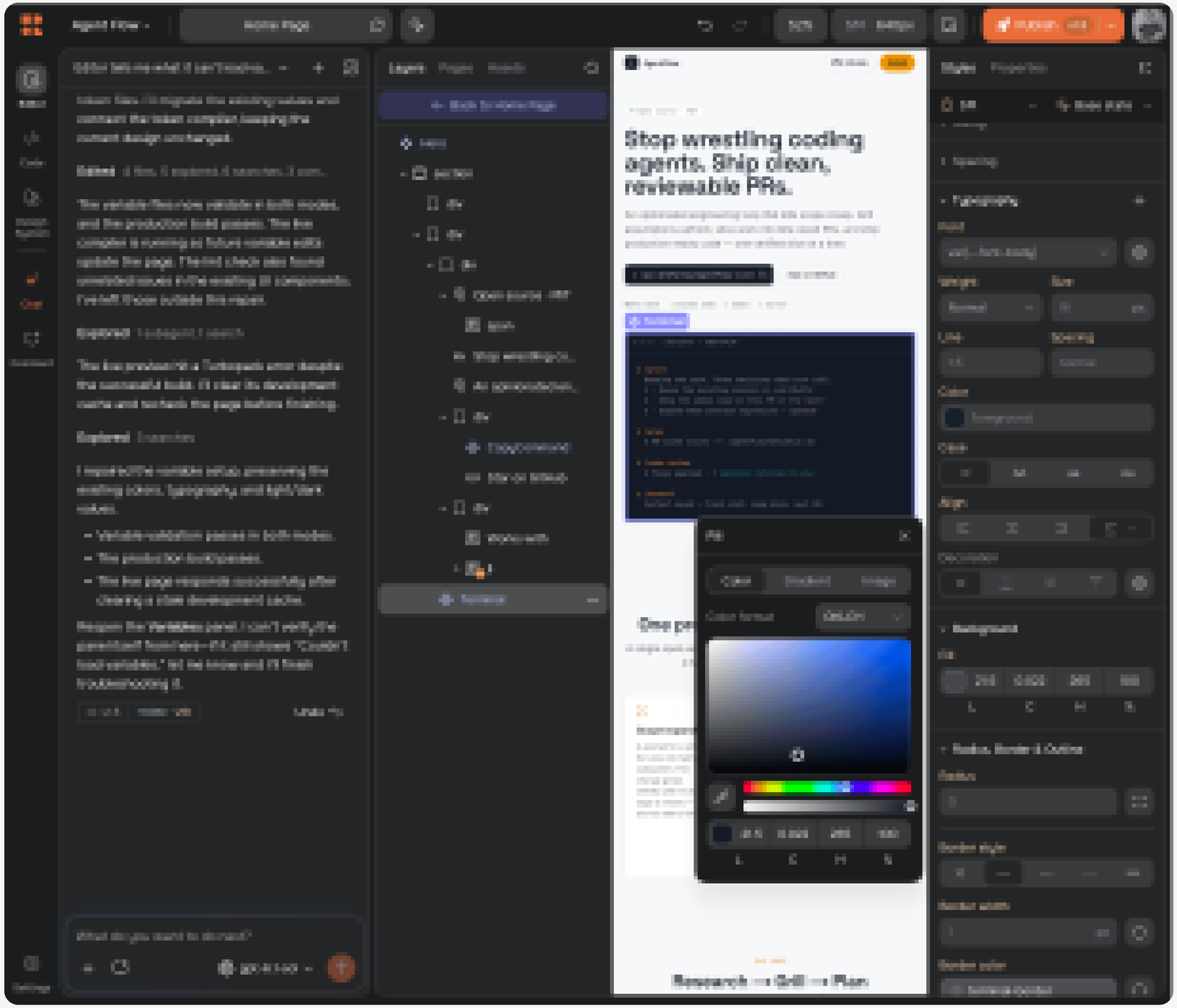
linkedin.com/in/kachurun

Why

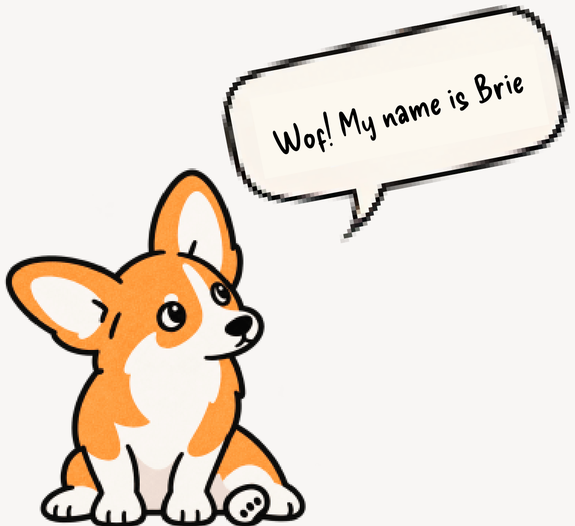
Как я тут оказался

Последний год я разрабатываю продукт вокруг собственного coding-агента уровня Claude code и хочу поделиться опытом

reforma



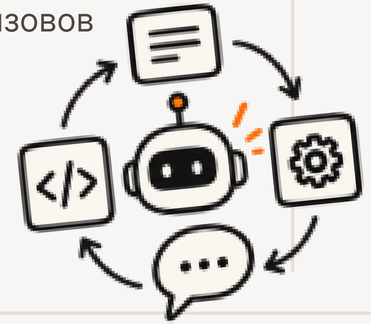
КАРТА ДОКЛАДА



01

ЦИКЛ УМНОЖАЕТ СТОИМОСТЬ

- Один запрос – много вызовов
- Модель под задачу
- Кеш и платные функции



02

КОНТЕКСТ РАСТЁТ

- Конфигурация кеша
- Стабильный префикс
- Сжатие истории

03

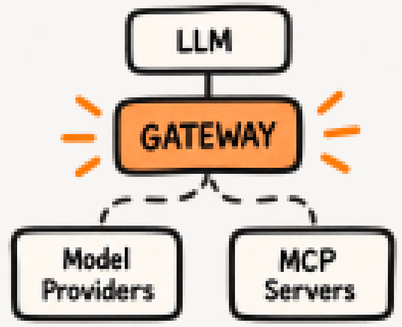
TOOLS СЪЕДАЮТ ТОКЕНЫ

- Загружаем только нужные
- Сжимаем результаты

04

ОБВЯЗКА РАСТЁТ

- Model Gateway
- MCP Gateway



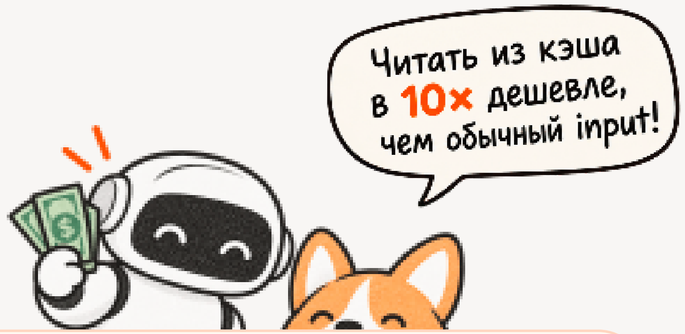
Агент это Цикл

Один запрос пользователя ≠ один запрос к модели

Тирания KV-Cache

Цена обработки одинакового объёма контекста (1M токенов)

Cached Input	$0.40	0.1x
Input (без кэша)	$4.00	1x
Cache write (запись в кэш)	$5.00	1.25x
Output (генерация)	$20.00	5x



Изменил префикс →
вместо $0.4 платишь $4

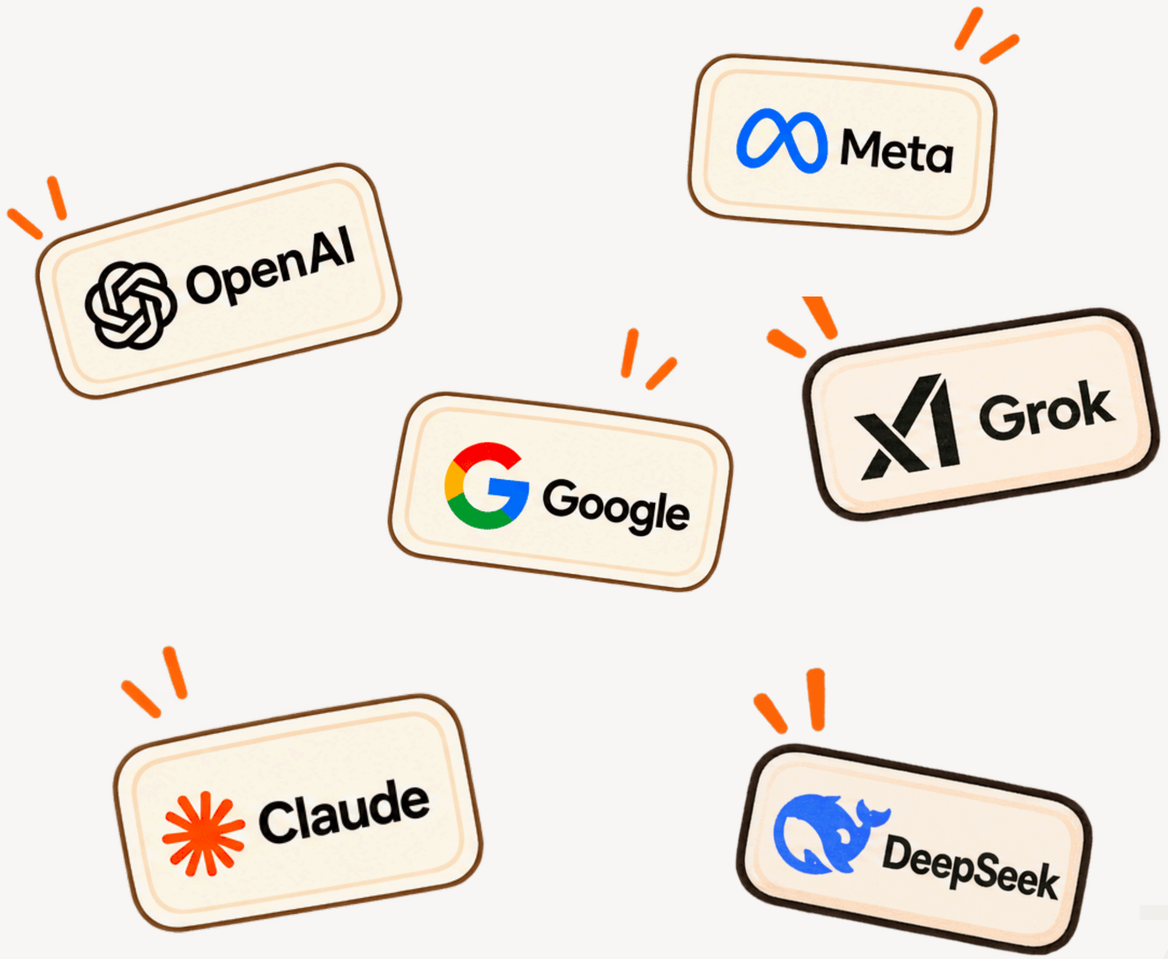
cached input, $0.4 / 1m tokens

↓ Изменили что-то в начале...

uncached input, $4 / 1m tokens

Как платить меньше за решённую задачу?

Выбор модели для правильного
баланса цены и качества

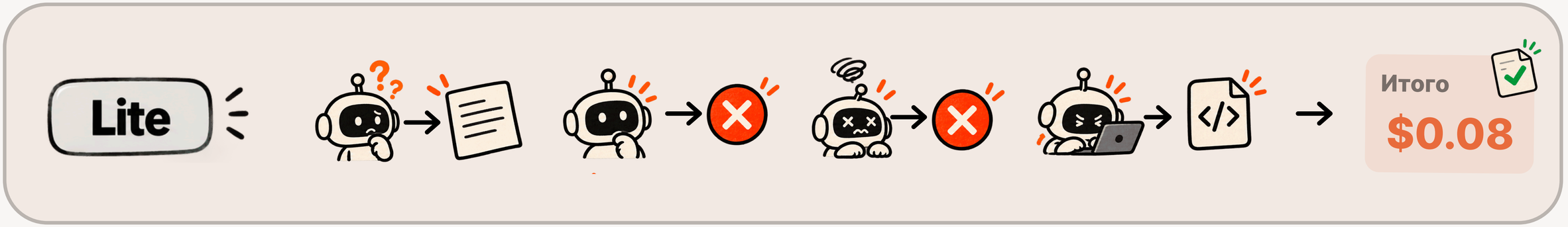


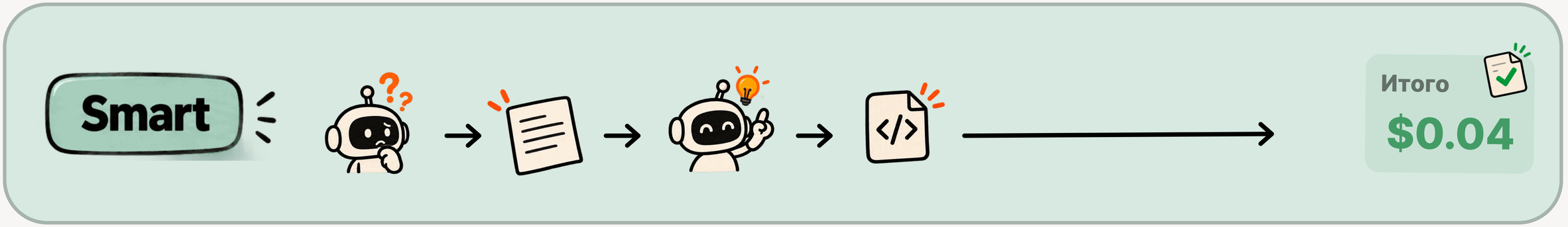
1. Насколько модели сложно и сколько она размышляет для решения задачи
2. Стабильность кеша и эффективная стоимость Input
3. Возможности модели и стоимость доп. функций

1

Модель дешевле – задача дороже

Слабая модель больше думает, ошибается и повторяет для выполнения той же задачи





2

Разная скидка и стабильность кеша

Стоимость модели может быть выше, но щедрая скидка это компенсирует

Model 1

Input	$2.00
Cached Input -75%	$0.5
Cache Hit Rate (avg)	80%
Effective Input	$0.80

96k токенов (cached) $0.05

24k (fresh) $0.05

Итого (только input) **$0.10**

Model 2

Input	$4.00
Cached Input -90%	$0.4
Cache Hit Rate (avg)	95%
Effective Input	$0.58

114k токенов (cached) $0.046

6k (fresh) $0.24

Итого (только input) **$0.07**

3

Фичи провайдера стоят денег

Некоторые провайдеры имеют очень приятный функционал

Модель A

Дорогая модель, дешёвые фичи

Модель	**$$$**
Встроенные фичи	**$**

Web Search	✓
File Search	✓
Compaction	✓
Другие фичи	✓

Модель B

Дешёвая модель, дорогие фичи

Модель	**$$**
Встроенные фичи	**$$$**

Web Search	✓
File Search	✓
Compaction	✓
Другие фичи	✓

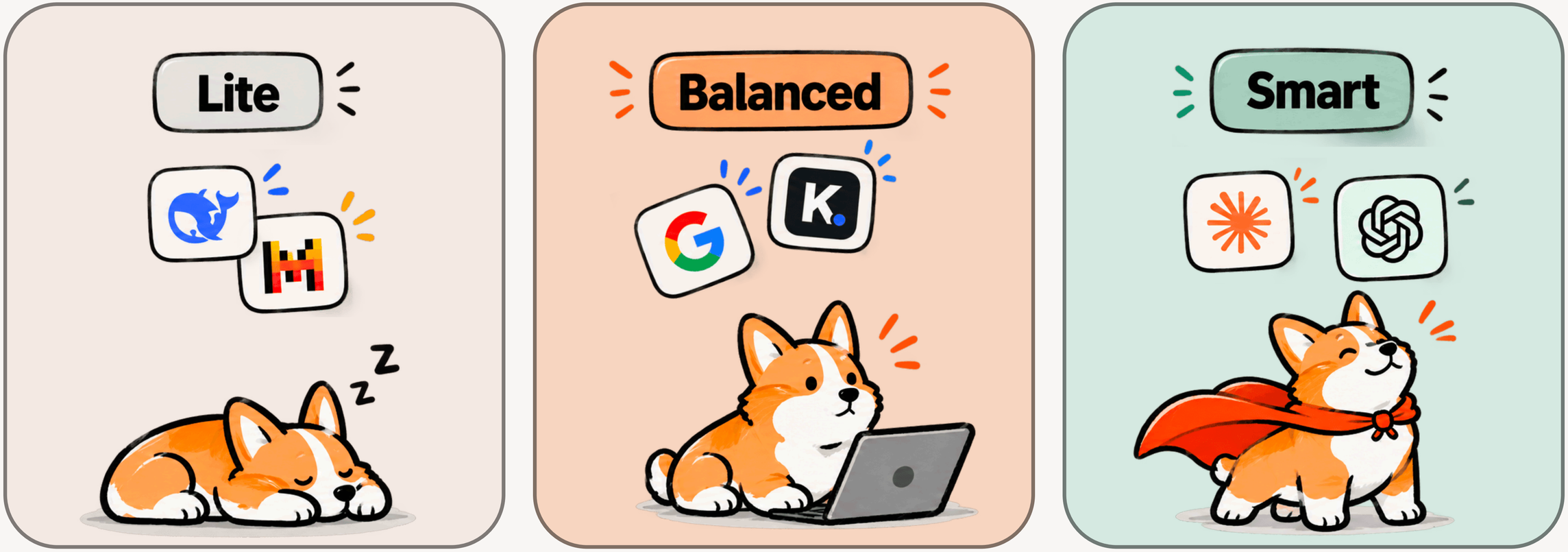
Модель C

Дешёвая модель, нет готовых фич

Модель	**$**
Встроенные фичи	**—**

Web Search	✗
File Search	✗
Compaction	✗
Другие фичи	✗

Выбираем грейд, а не модель



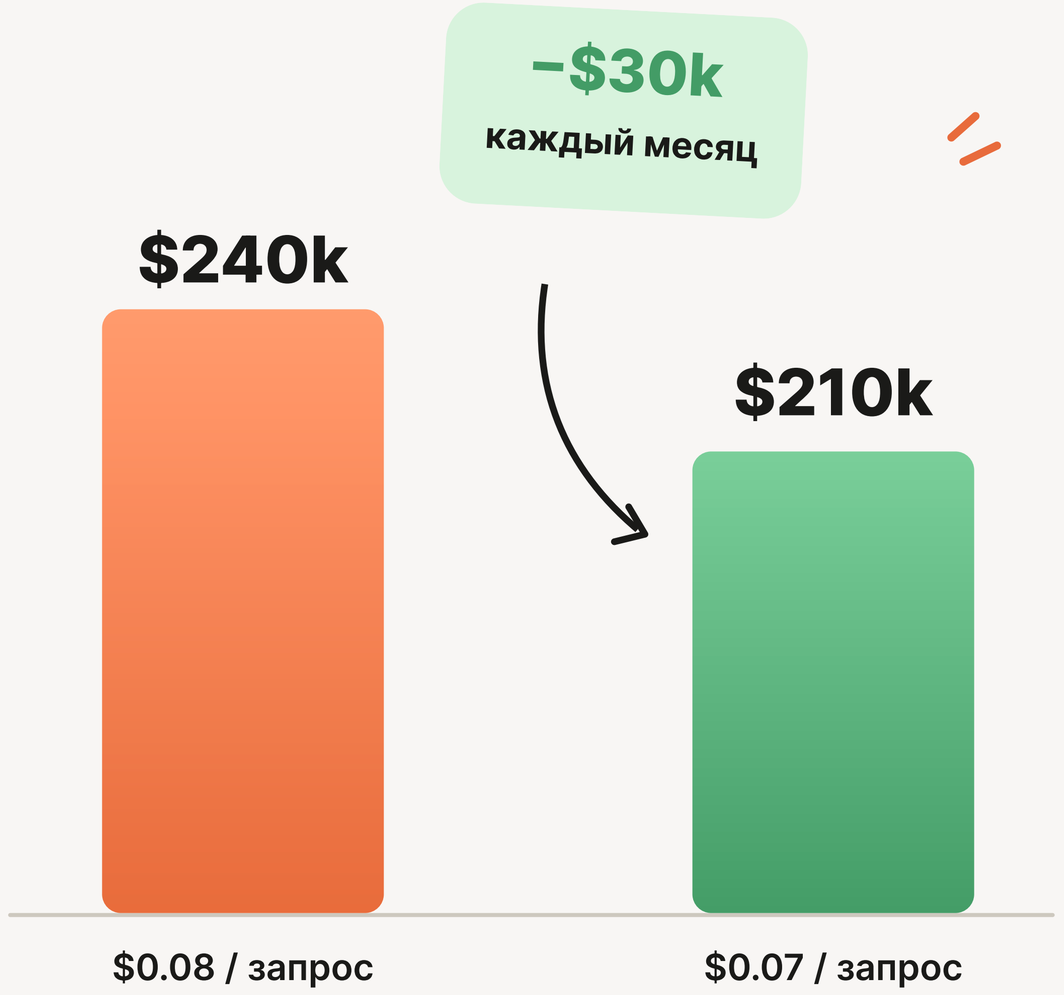
Каждая копейка на счету

Один пользователь делает десятки или сотни запросов в день.
Тысяча пользователей – 3 млн запросов в месяц.

Параметры расчёта

01	Пользователей	~1 000
02	Запросов на пользователя	~100 / день
03	Всего запросов	~3 млн / месяц
04	Спред цены	$0.01 / запрос

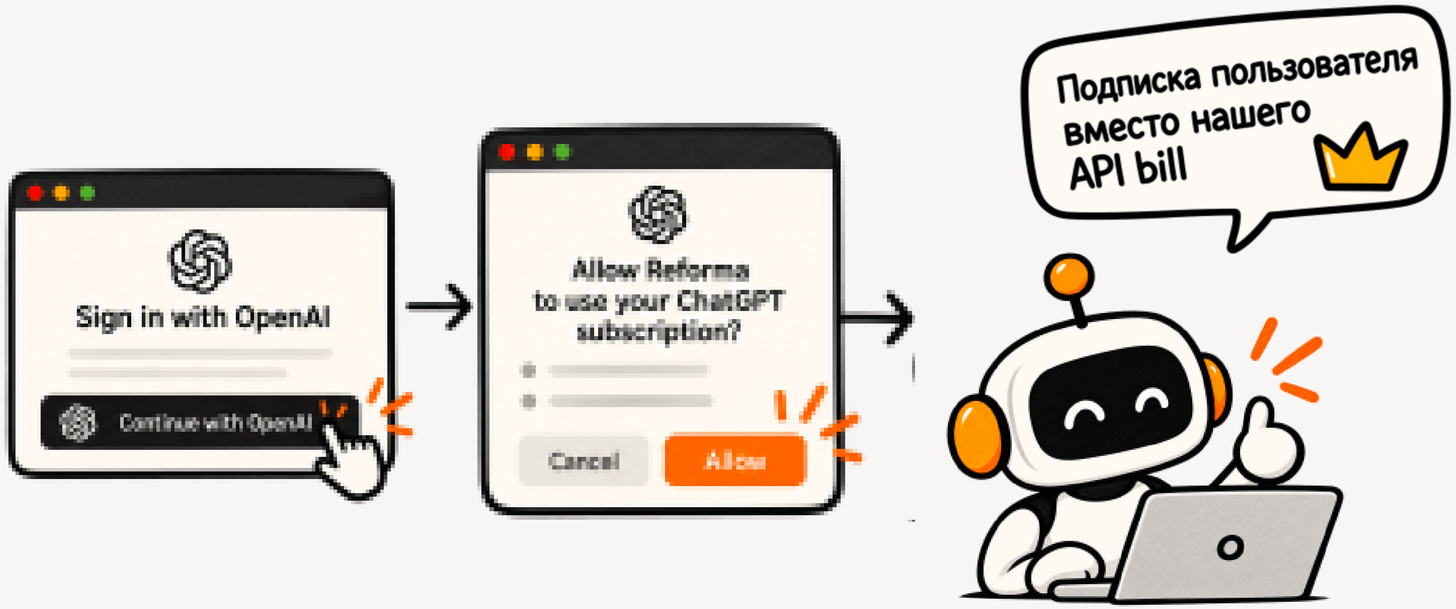
3 млн × $0.01 = $30 000 / месяц



BYAK

01 СВОЙ API KEY

02 ПОДПИСКА ПОЛЬЗОВАТЕЛЯ



$20 подписка пользователя дает аналог $700 наших расходов на API

Как платить меньше за решённую задачу?

01 **Считаем стоимость решённой задачи**
Учитываем reasoning, повторы и число вызовов модели.

02 **Выбираем грейд под сложность задачи**
Lite, Balanced или Smart – для разных сценариев

03 **Учитываем кеш и функции провайдера**
Cache hit и платные tools меняют итоговую стоимость.

04 **Подключаем доступ пользователя**
API key или подписка пользователя снижают расходы почти до нуля

Cache miss делает задачу в разы дороже

Каждый следующий вызов получает растущий контекст.
Цена зависит от того, попал ли этот контекст в кеш.

Вызов A

180k контекста в кеше → 90% CACHE HIT

$0.108
20k input + write, 180k cached input

Вызов B

20k контекста в кеша → 10% CACHE HIT

$0.812
180k input + write, 20k cached input

gpt-6.1-sol · 1 turn · 200k context · x7.5 дороже

Почему не работает кеш?

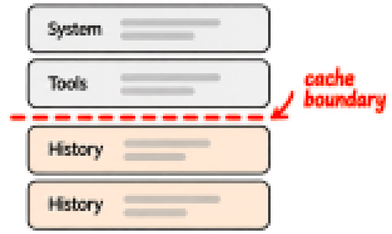
Проблема в конфигурации или семантике провайдера

1 Кеш не включён

Флаг может быть выключен или настройка не поддерживается выбранным provider

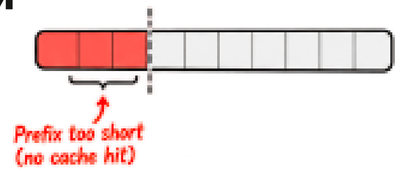
2 Boundary стоит не там

Cache breakpoint должен отделять стабильный префикс от меняющейся части запроса



3 Префикс слишком короткий

Он может не достигать минимального порога provider и поэтому вообще не попадать в кеш



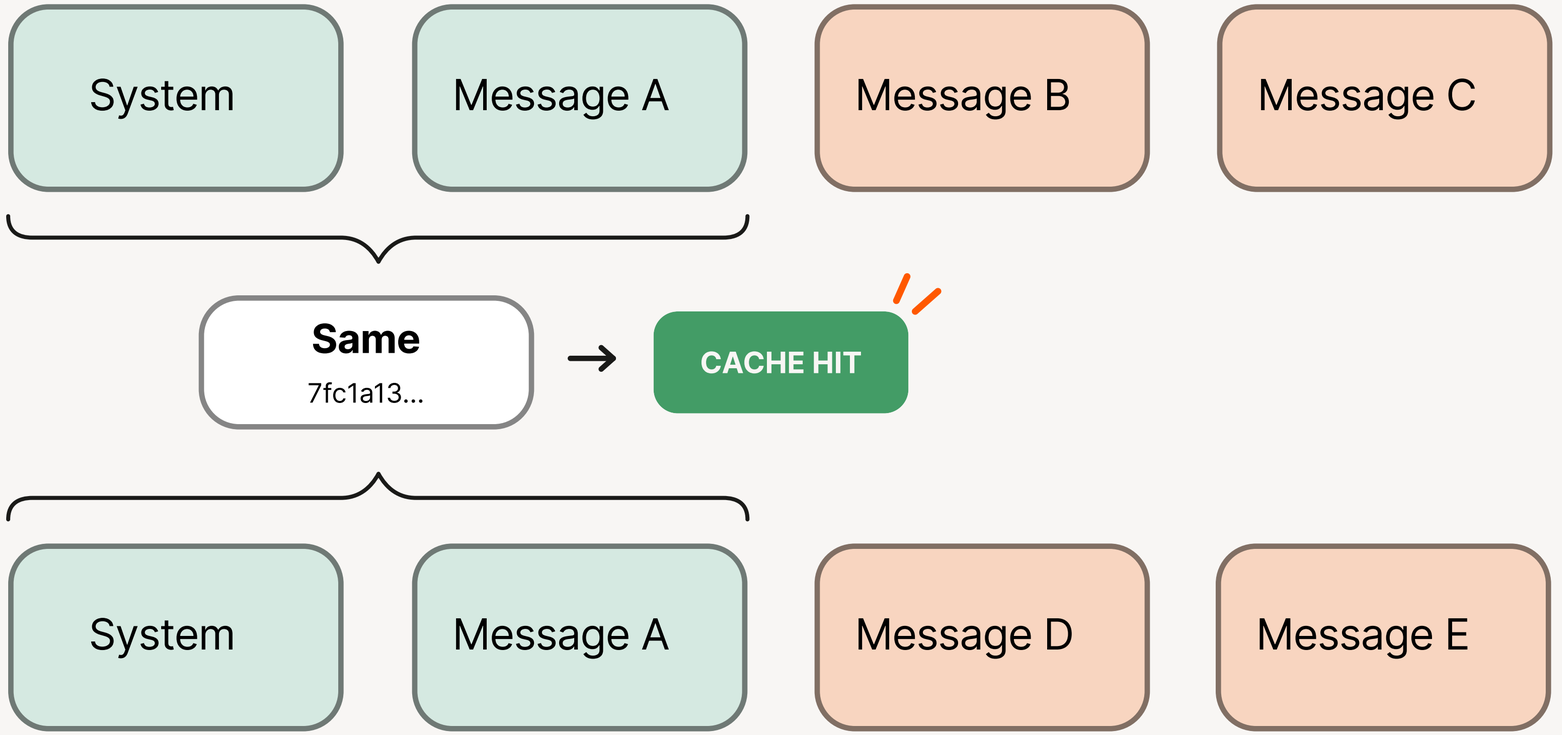
4 Cache entry уже истёк

Даже правильная настройка не поможет, если TTL вышел до следующего похожего запроса

Implicit Cache

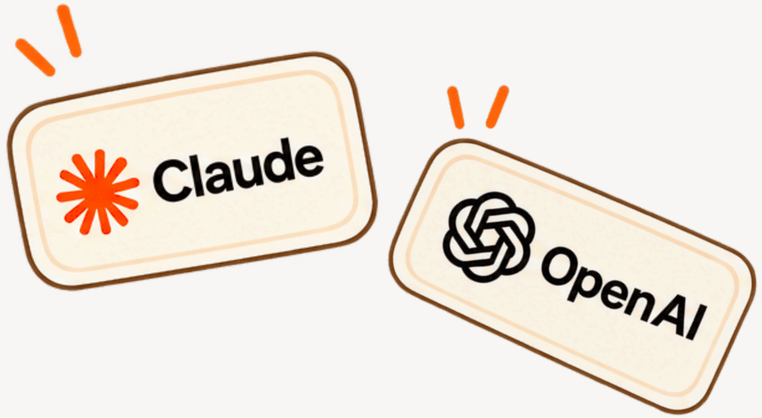
OpenAI Google DeepSeek

Совпадающая часть префикса – автоматический Cache hit



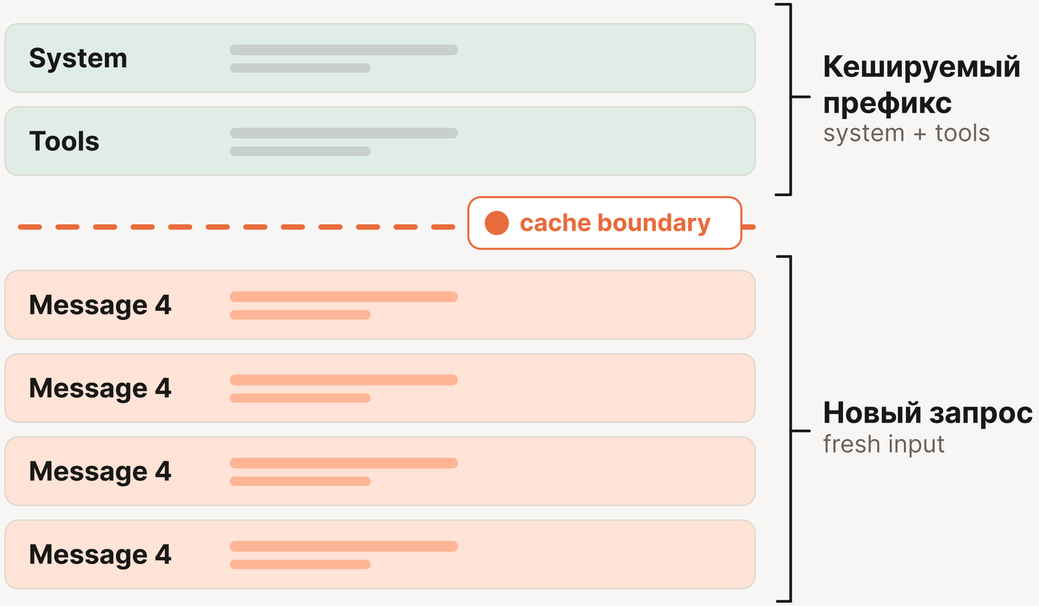
Explicit Cache

Кеш работает только если явно поставить метку.
Где поставить boundary — решаете вы.



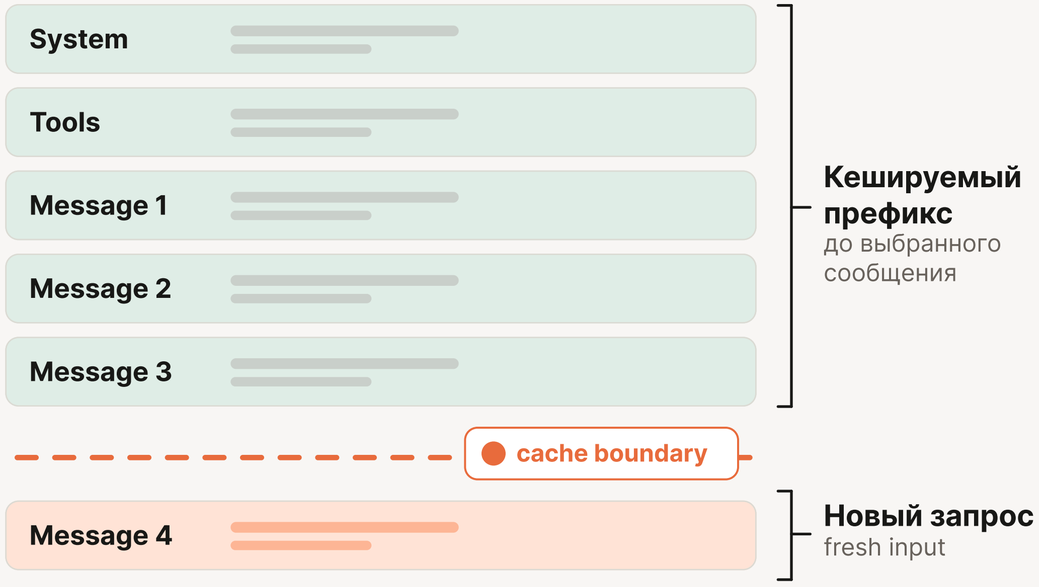
1 Boundary на system

Кешируется префикс до system



2 Boundary на сообщении

Кешируется до указанного сообщения

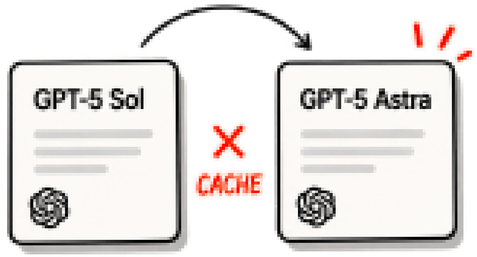


Теряется кеш?

Кеш настроен правильно, но стабильный префикс меняется между запросами.

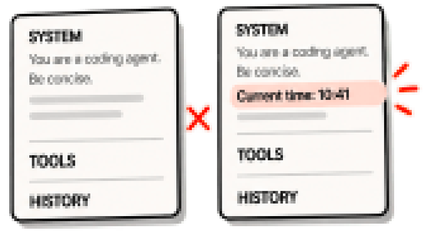
1 Смена модели / inference

Даже внутри одного provider разные модели и роуты могут уходить на разные inference — общего кеша между ними нет.



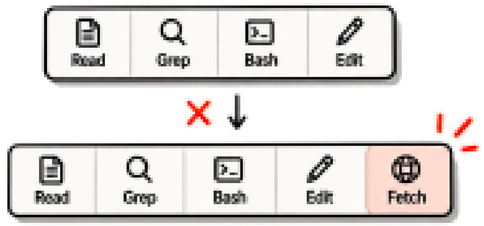
2 Динамический system prompt

Время, контекст пользователя и другие динамические данные меняют system prompt — cache key больше не совпадает.



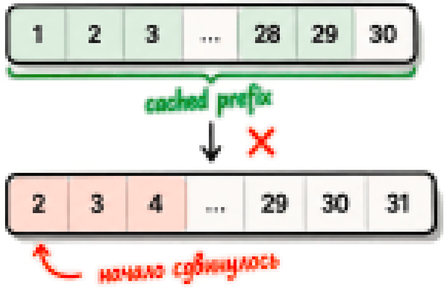
3 Меняется набор tools

Добавили, удалили или переставили инструмент — конфигурация tools изменилась, и прежний кеш больше не подходит.



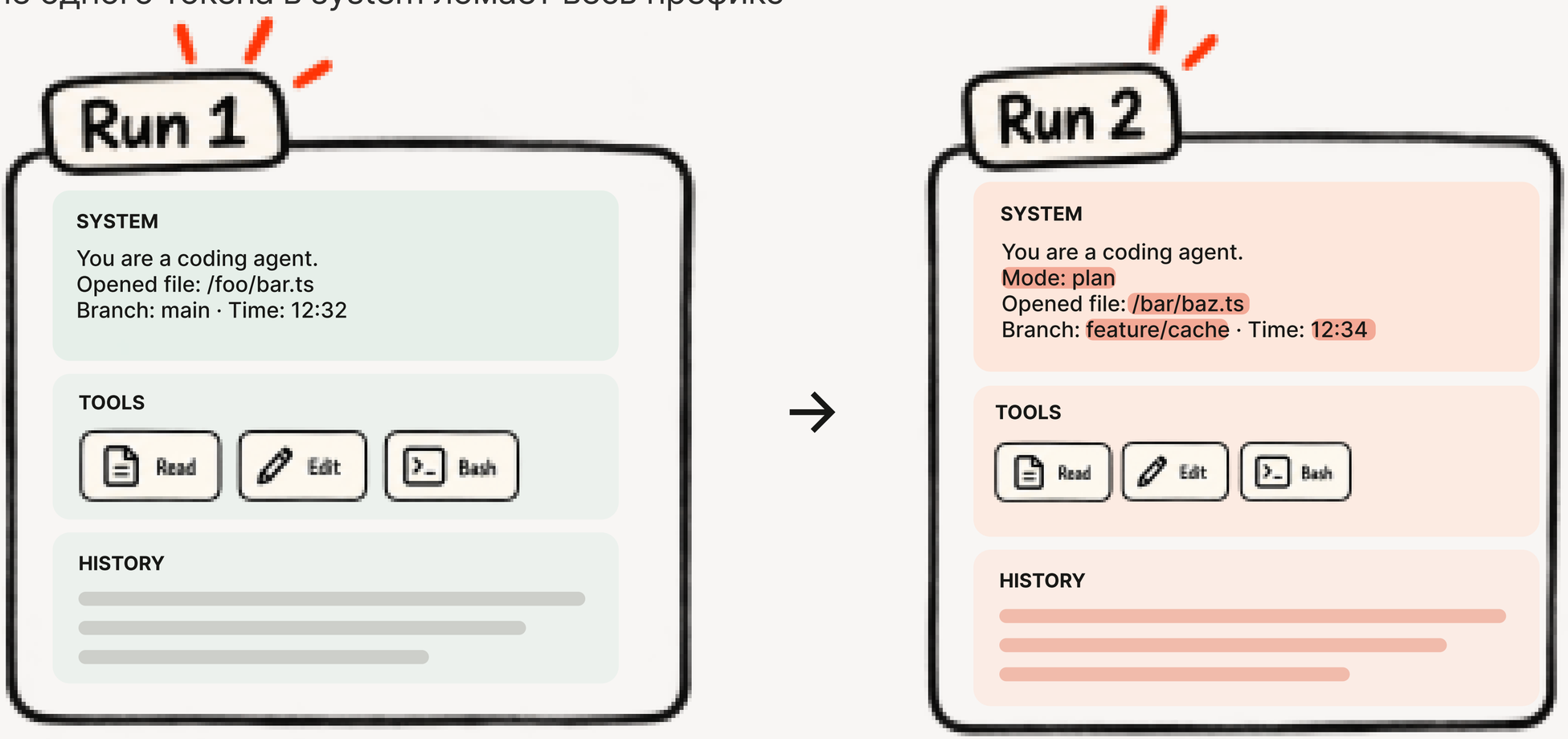
4 Обрезаем старую history

При sliding window начало history сдвигается. Стабильный префикс меняется — кеш перестаёт совпадать.



Динамические инструкции

Изменение одного токена в system ломает весь префикс



Динамику добавляем в конец

Расширяем сообщение пользователя свежим контекстом

Что видит пользователь

Исправь failing test

Сообщение...

Что получает модель

<user_query>
Исправь failing test
</user_query>

System

<context>
Current time: 12:43
Opened file: /src/app.spec.ts
Git branch: feature/pricing
Pending todos: 2
</context>

```
function toModelInput(
  userMessage: string,
  context: RuntimeContext,
  model: Model,
) {
  if (model.capabilities.allowSystemInMessages) {
    return [{
      role: 'system',
      content: formatContext(context),
    }, {
      role: 'user',
      content: userMessage,
    }]
  }

  // Fallback
  return [{
      role: 'user',
      content: `
        <context> ${formatContext(context)} </context>
        <user_query> ${userMessage} </user_query>
      `
  }]
}
```

FALLBACK
Если mid-turn system запрещён - добавляем <context> в копию user message.

Комбинируем контекст и инструменты

01

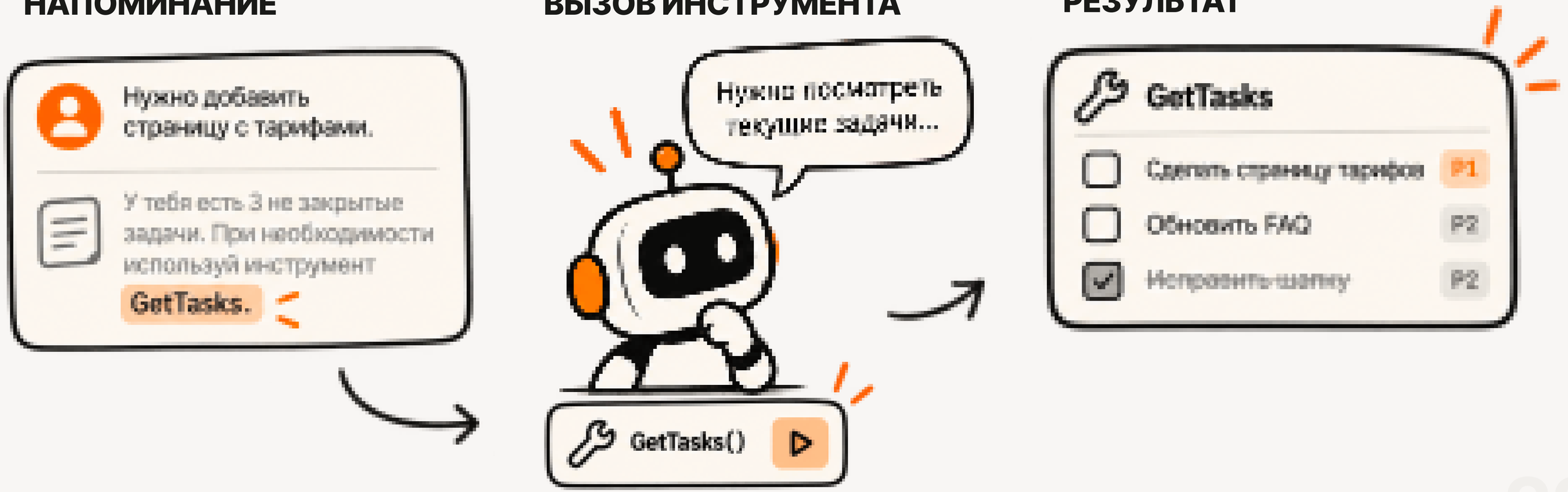
НАПОМИНАНИЕ

02

ВЫЗОВ ИНСТРУМЕНТА

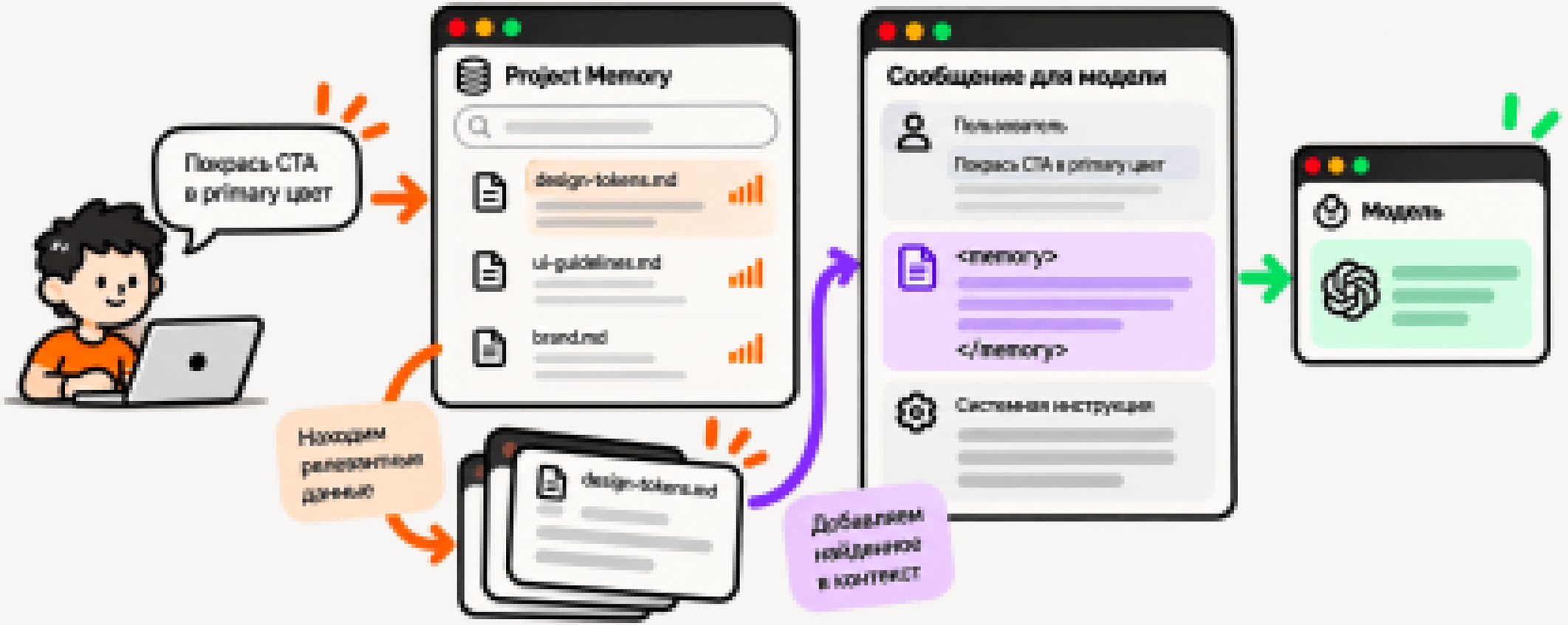
03

РЕЗУЛЬТАТ



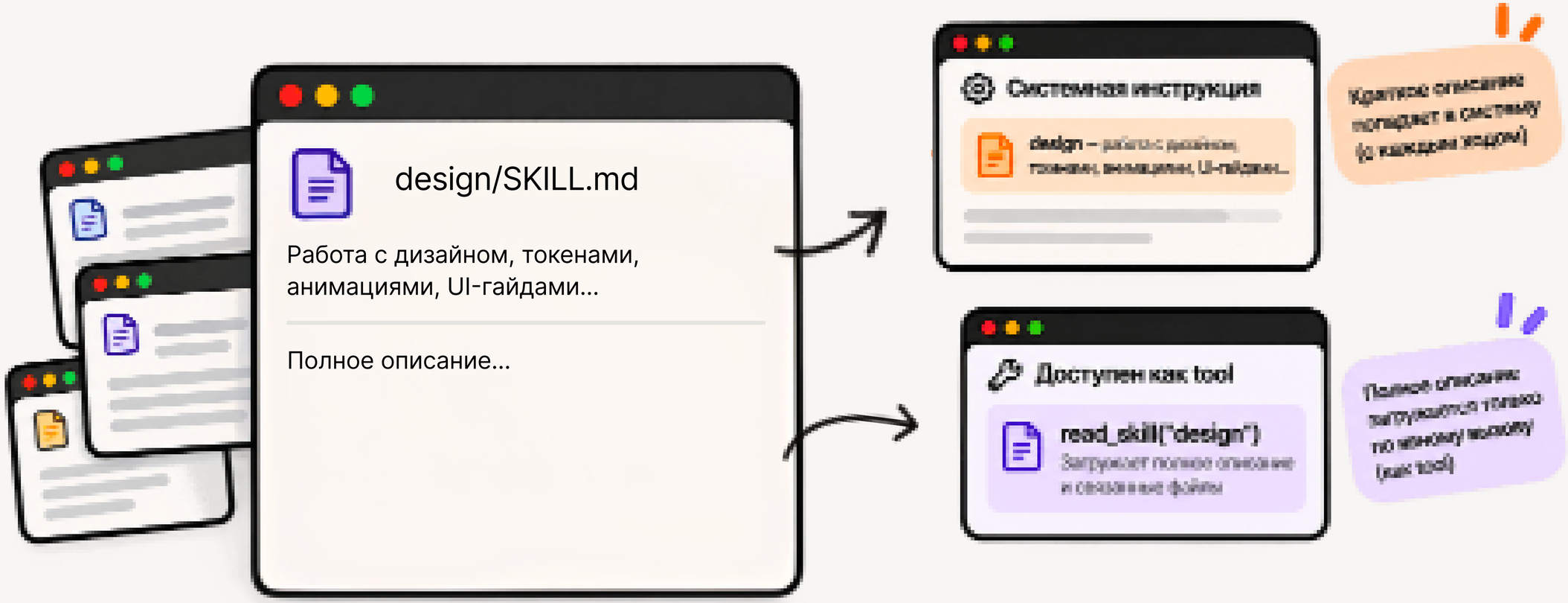
Подмешивание контекста к сообщению пользователя

Пример, подмешиваем плагин если текст содержит определенные слова



Skills – знания по требованию

В базовом контексте оставляем только короткое описание доступных skills, а полный skill модель получает только когда он реально нужен.



Контролируем содержимое контекста

01 **Проверьте, что кеш действительно работает**

Проверяйте hit rate, boundary, TTL и минимальный размер префикса

02 **Сохраняйте префикс стабильным**

Не меняйте модель, system, tools и старую историю без причины

03 **Динамику добавляйте в конец**

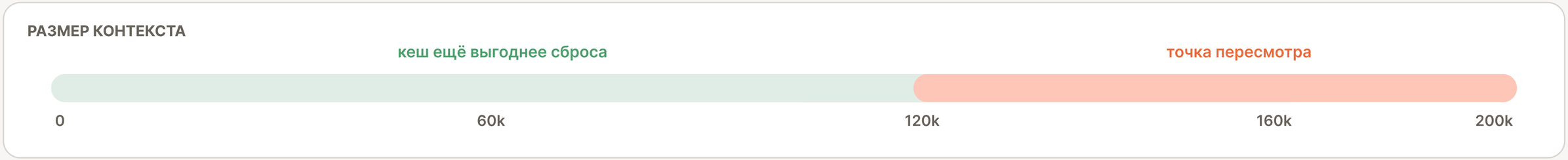
Новый контекст — в хвост, обогащённые сообщения сохраняйте

04 **Знания загружайте по требованию**

Короткое напоминание → tool или skill только когда нужны

Лимит контекстного окна

Кеш снижает цену стабильного префикса, но не отменяет предел контекстного окна



01

Дороже

Даже cached input продолжает стоить денег. После порога выгоднее начать новый контекст.

PRICE ↑

02

Медленнее

Большой input увеличивает prefill и время каждого следующего шага агента.

LATENCY ↑

03

Хуже качество

Слабые модели начинают забывать детали, путаться и сильнее галлюцинировать.

QUALITY ↓

Summarization

Три способа сжать историю + сжатие на стороне провайдера

01

TRIM

Оставить последние N сообщений

Мгновенно и бесплатно

CODE PATH

02

MICROCOMPACT

Сохранить структуру, убрать шум

Без LLM и ожидания

CODE PATH

03

LLM SUMMARY

Получить пересказ по шаблону

Лучшее качество, но есть latency

MODEL PATH

04

PROVIDER SUMMARY

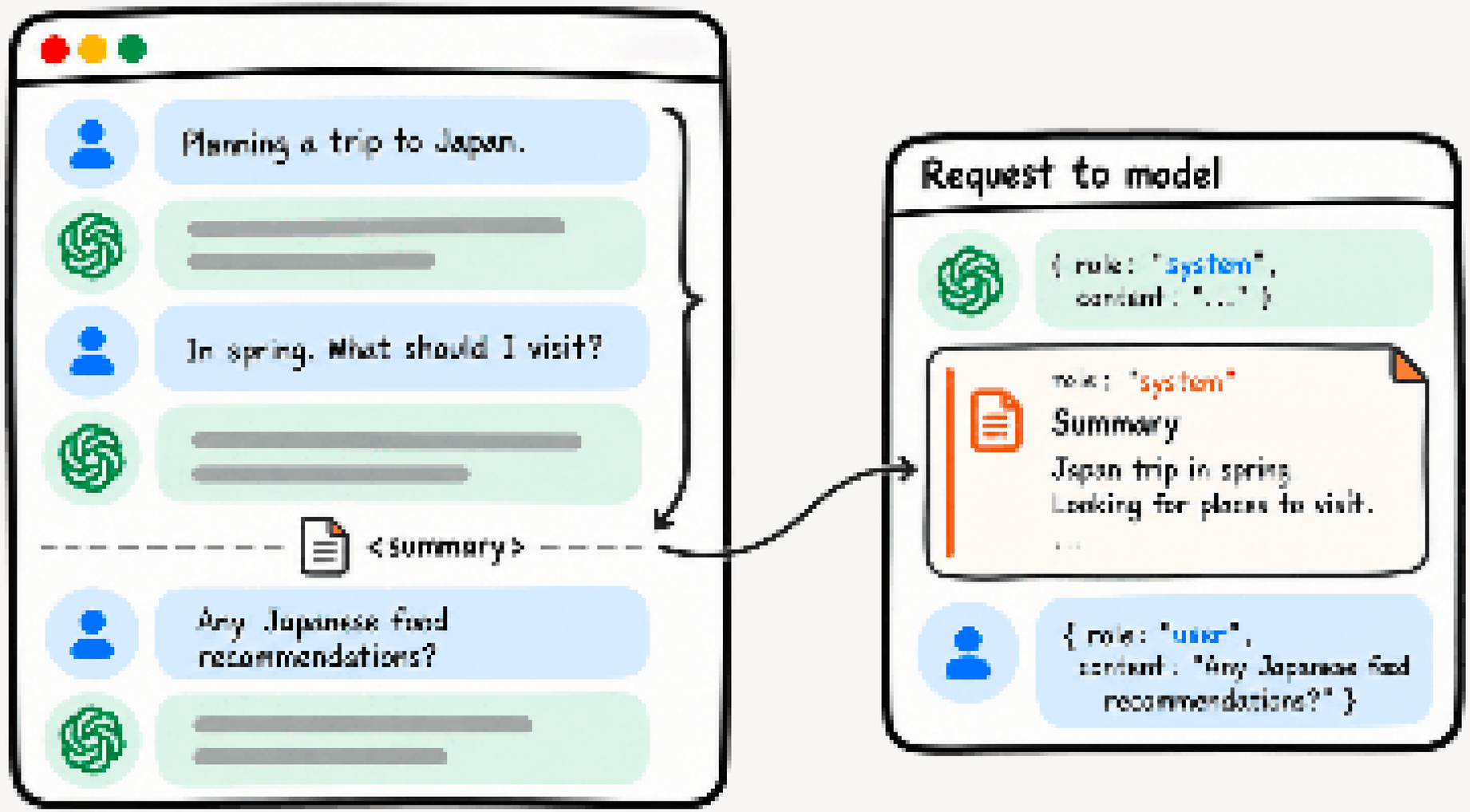
Использовать встроенную compaction

Минимум кода

MODEL PATH

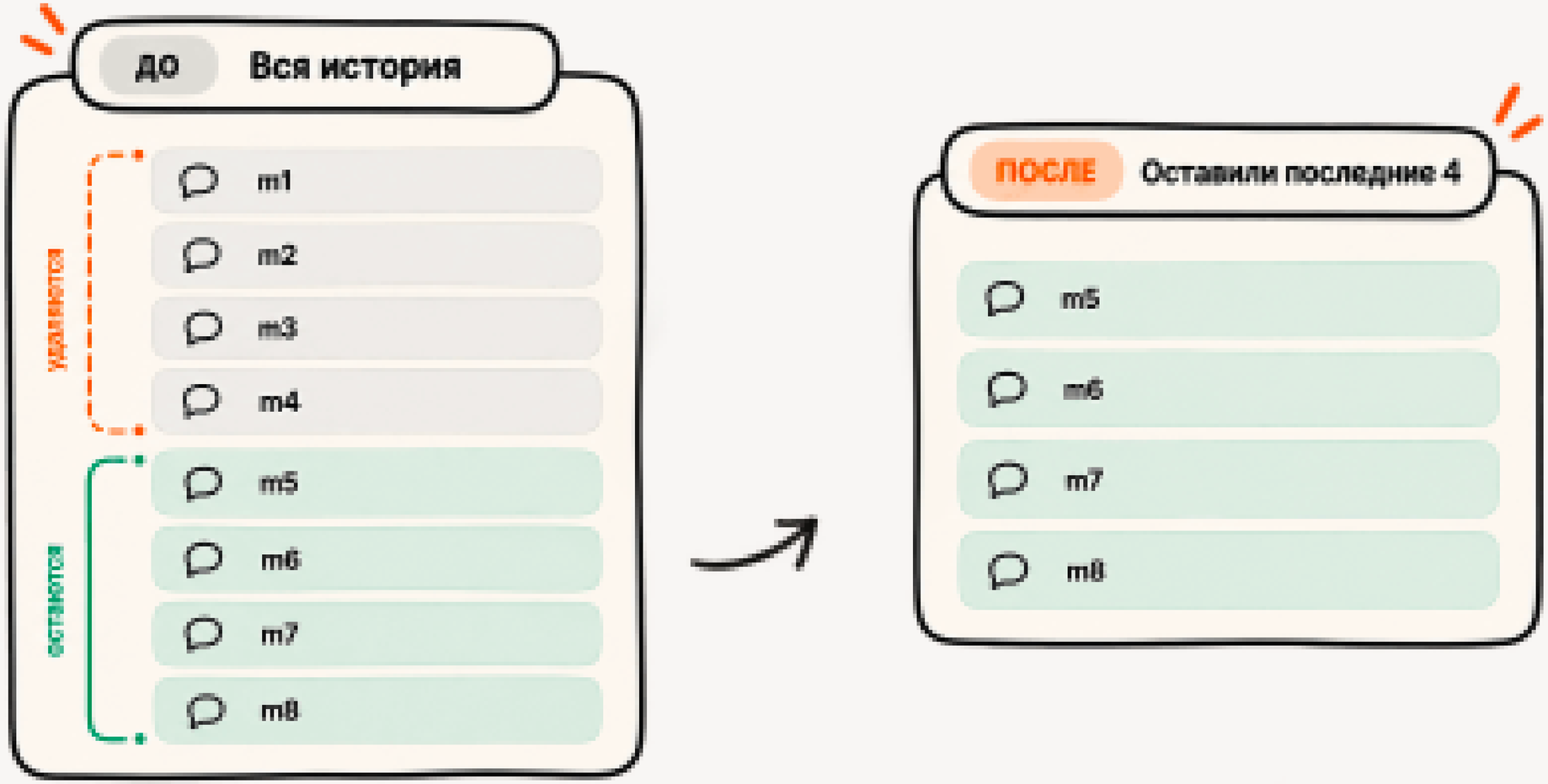
Summary <marker>

UI остаётся прежним, но между сообщений хранится скрытый объект содержащий сжатую историю

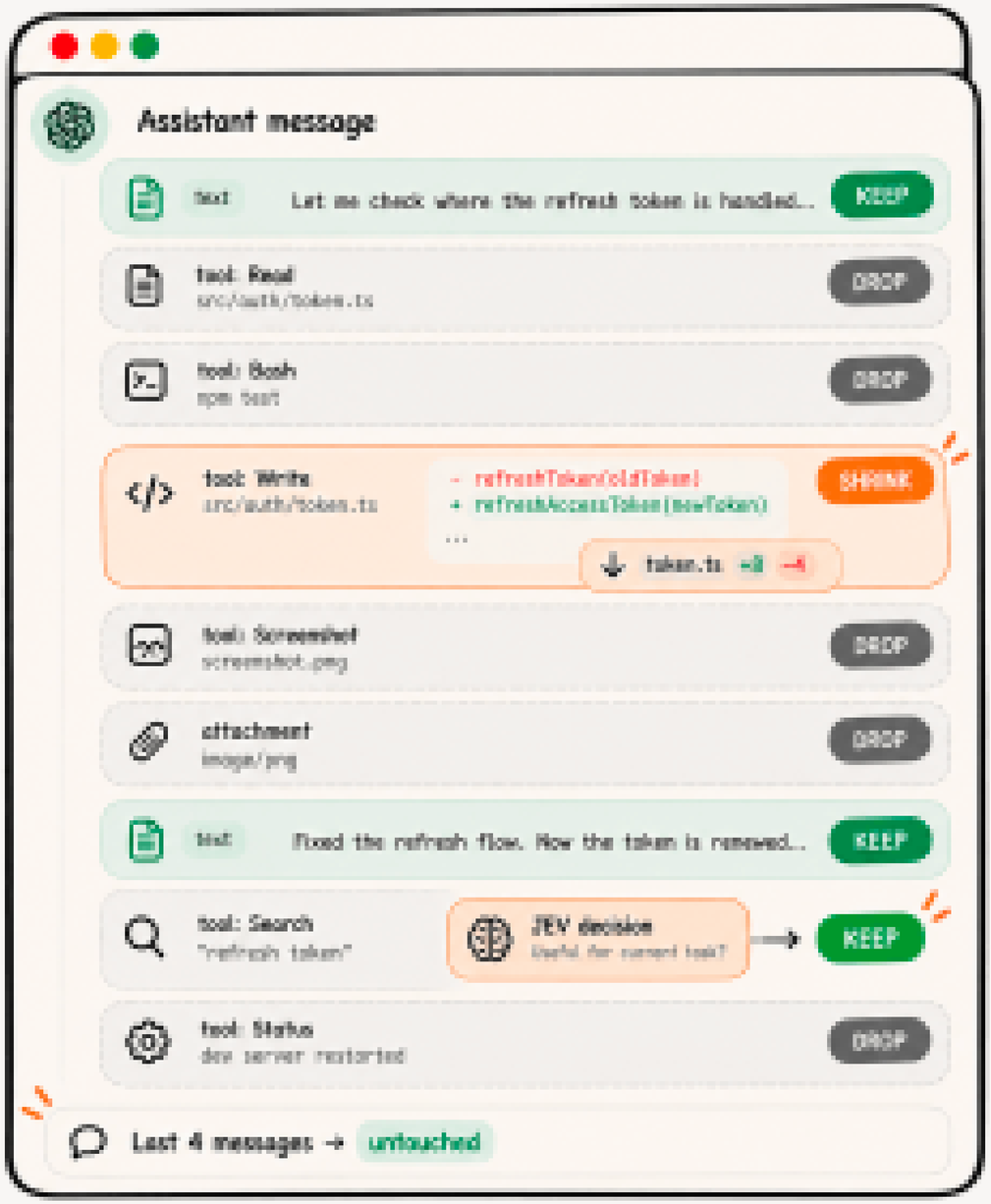


Trim

Самый простой подход – отправляем только хвост



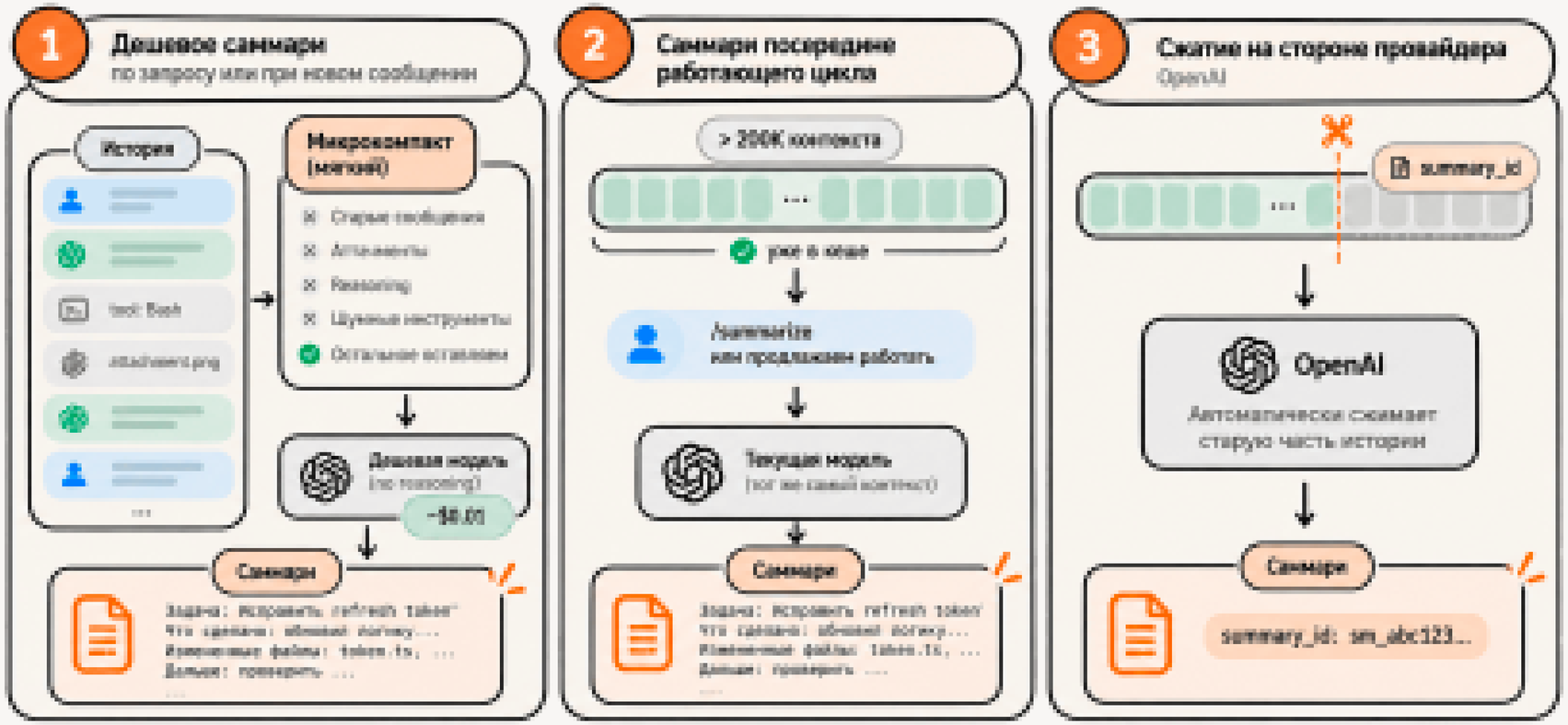
Microcompact

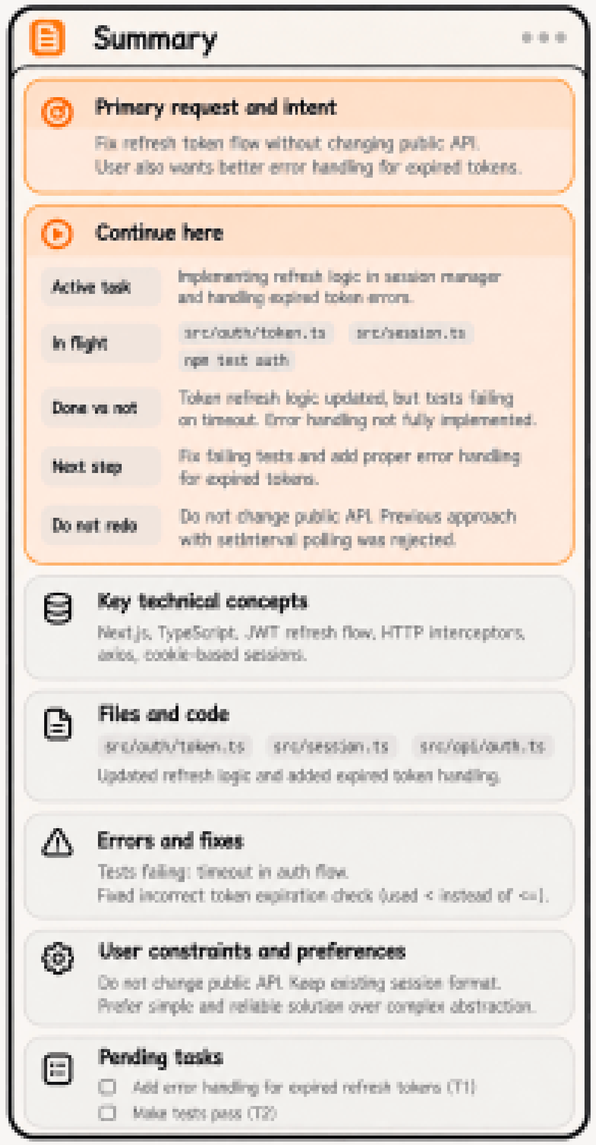


До 90% контекста — мгновенно,
детерминированно и бесплатно

LLM Summarization

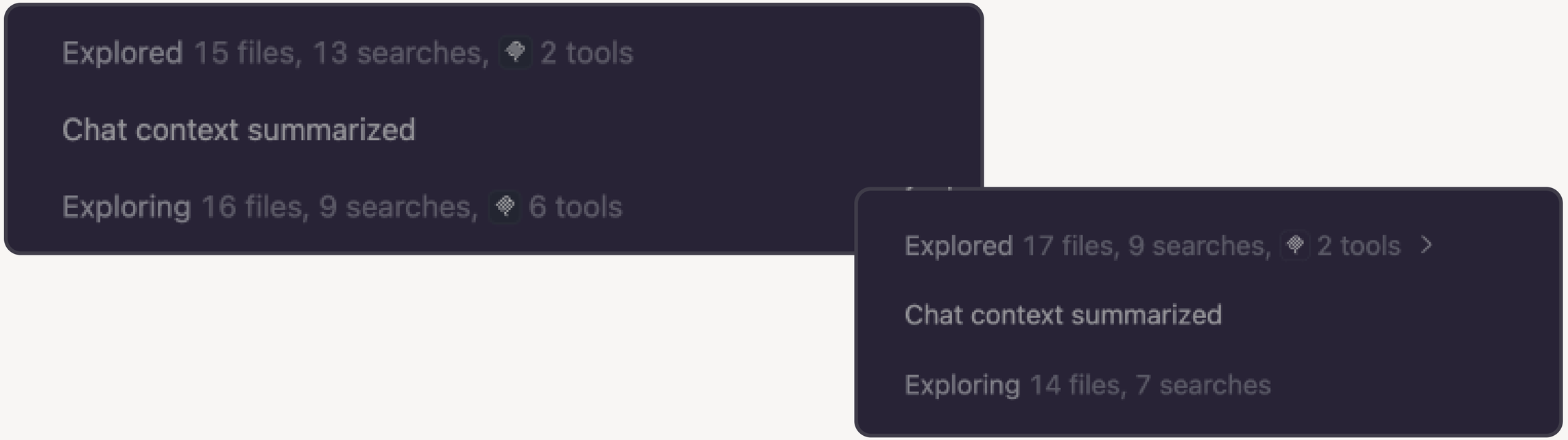
Формат summary задаём сами. Отличается только момент запуска и выбранный исполнитель.





Минусы сжатия

Если запустить его посередине активной задачи, модель может забыть работу и начать восстанавливать контекст



Когда запускать сжатие

Порог зависит от момента: новое сообщение пользователя или авария прямо внутри agent loop.

НОВОЕ СООБЩЕНИЕ ПОЛЬЗОВАТЕЛЯ

> 50 messages / 60k

Microcompact

Много сообщений и уже заметный объём.

> 160k

LLM summary

Жирный контекст: дальше гонять невыгодно.

> 120k / 30min+

Microcompact

Старый чат: шанс cache hit почти нулевой.

/summarize

Ручной запуск

Пользователь сам решает зафиксировать состояние.

МЕЖДУ TOOL CALLS

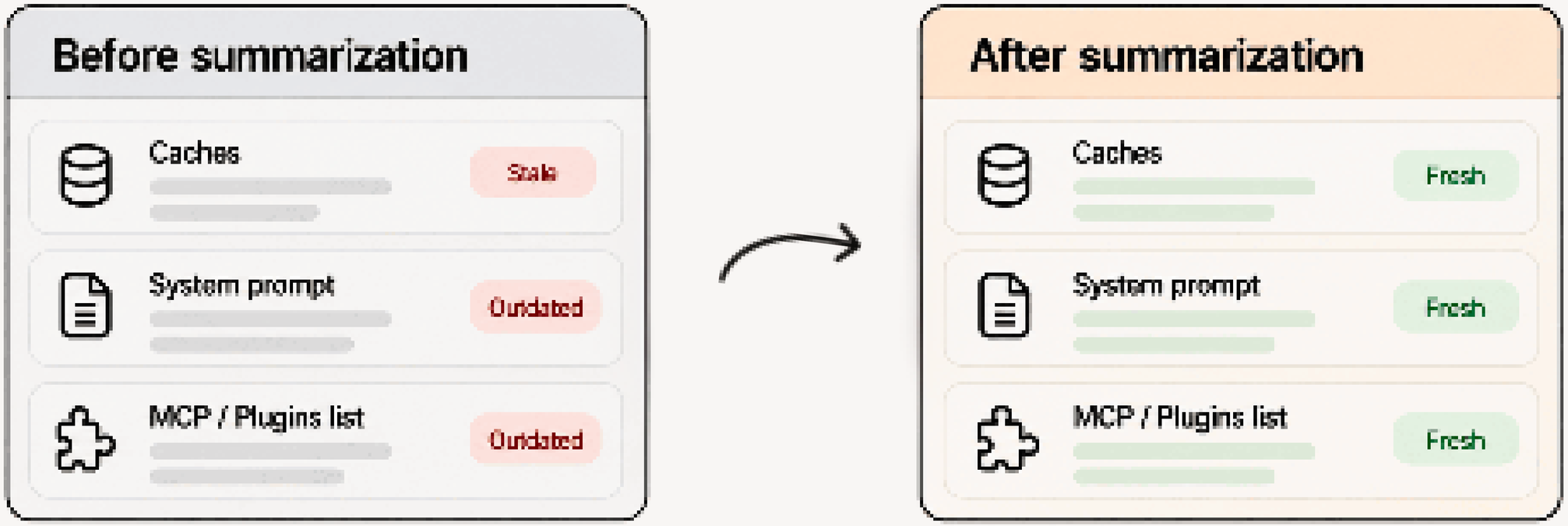
200k+

EMERGENCY

Контекст уже не помещается. Сжимаем немедленно, даже если текущий ход ещё не закончен.

Точка сброса состояния

Кеш уже потерян. Используем этот момент, чтобы пересобрать всё динамическое окружение



Сжимаем, не теряя задачу

01 **Сначала – microcompact**

Быстро убираем очевидный шум

02 **LLM summary – только по порогу**

Внутри agent loop – только аварийно

03 **После сжатия задача продолжается**

Момент используем для пересборки состояния

Десятки инструментов
налог на каждый ход

10+ инструментов – налог на каждый ход

Schemas попадают в system prompt и ездят по кругу, даже если инструмент не понадобится никогда

TOOL CATALOG

10+

инструментов агента

- Read
- GitHub
- Edit
- Figma
- Bash
- Linear
- Search
- MCP: ...
- Browser

→

SYSTEM PROMPT

instructions

+

tool schemas × N

name · description · parameters

ПОВТОРЯЕТСЯ НА КАЖДОМ ХОДЕ

→

ЭФФЕКТ

01 Контекст раздувается

Платим токенами и задержкой за неиспользуемые schemas.

02 Модель теряется в выборе

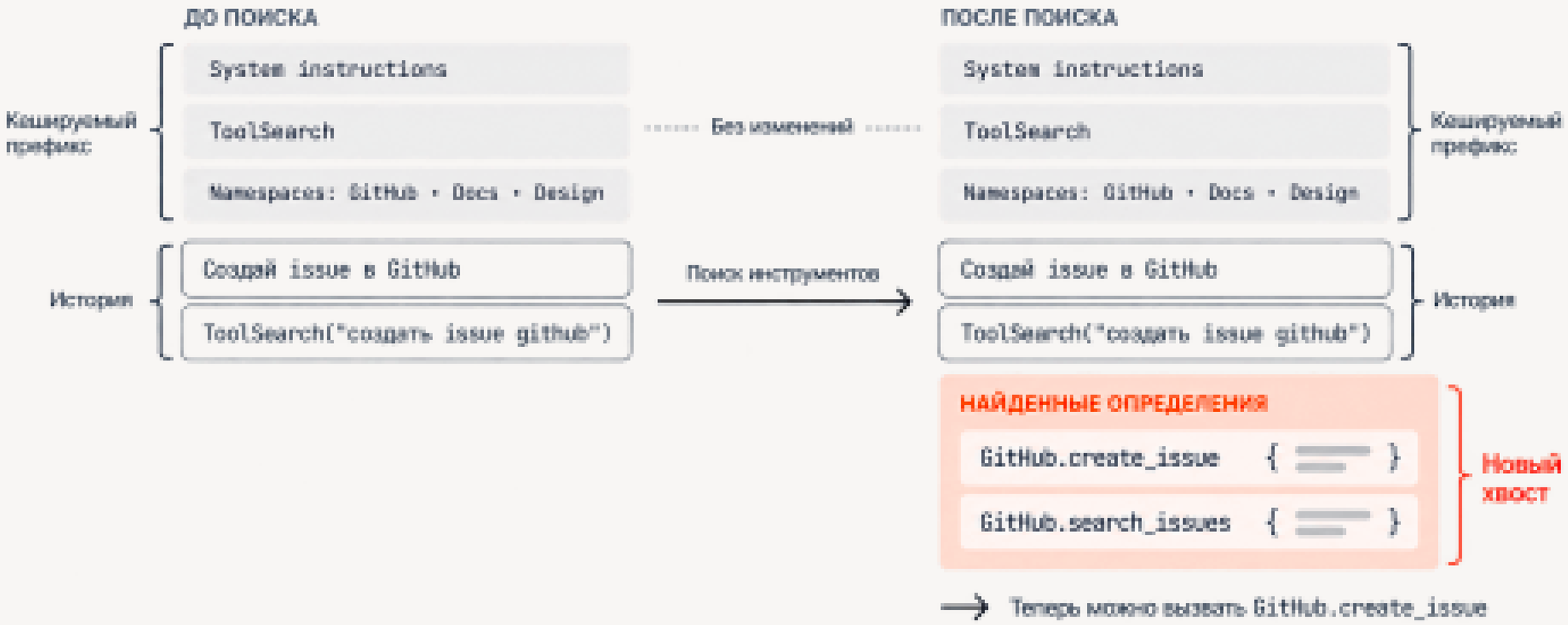
Качество выбора падает при десятках похожих tools.

03 MCP умножает масштаб

Несколько серверов легко превращают 50 tools в сотни.

ToolSearch Tool

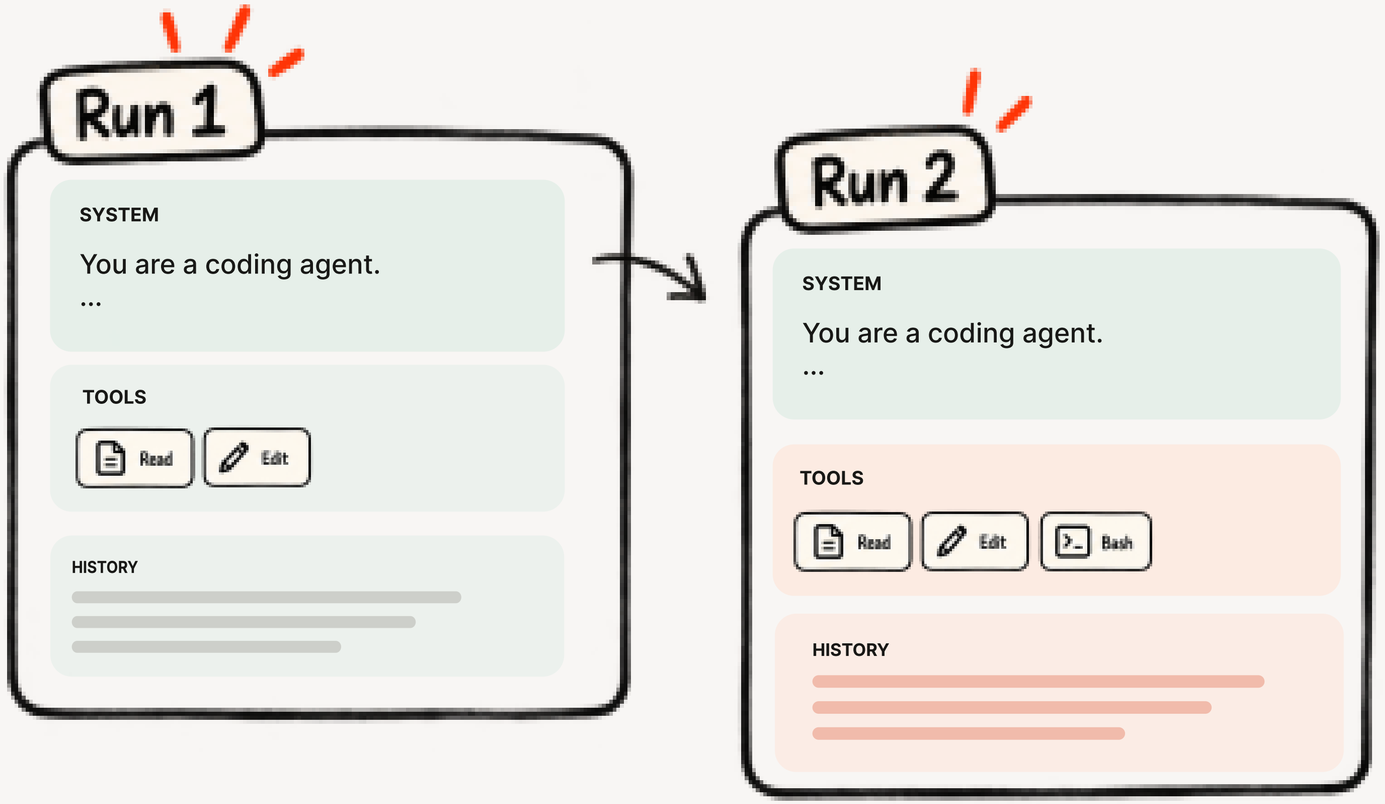
В system prompt остаётся ToolSearch, а определения появляются только в хвосте контекста



Динамическая смена Active tools

Новый список активных tools переписывает system prompt и инвалидирует весь prefix

```
generateText({
  model,
  tools: {
    read: readTool,
    edit: editTool,
    bash: bashTool,
  },
  prepareStep: (step) => ({
    activeTools: isBashLoaded
      ? ['read', 'edit', 'bash']
      : ['read', 'edit'],
  }),
});
```



ЭКСПЕРИМЕНТАЛЬНО

Описать tool обычным текстом

Модель понимает schema, даже если API не разрешает добавить объявление в хвост

01

Schema как текст

Имя, назначение и параметры добавляем в message.

```
tool: "search_docs"
args: { query }
```

02

Модель вызывает tool

Генерирует structured tool call.

03

SDK падает

Tool не объявлен – вызов считается битым.

NOT DECLARED

04

Repair handler

Перехватываем имя и args, проверяем registry.

recoverToolCall()

05

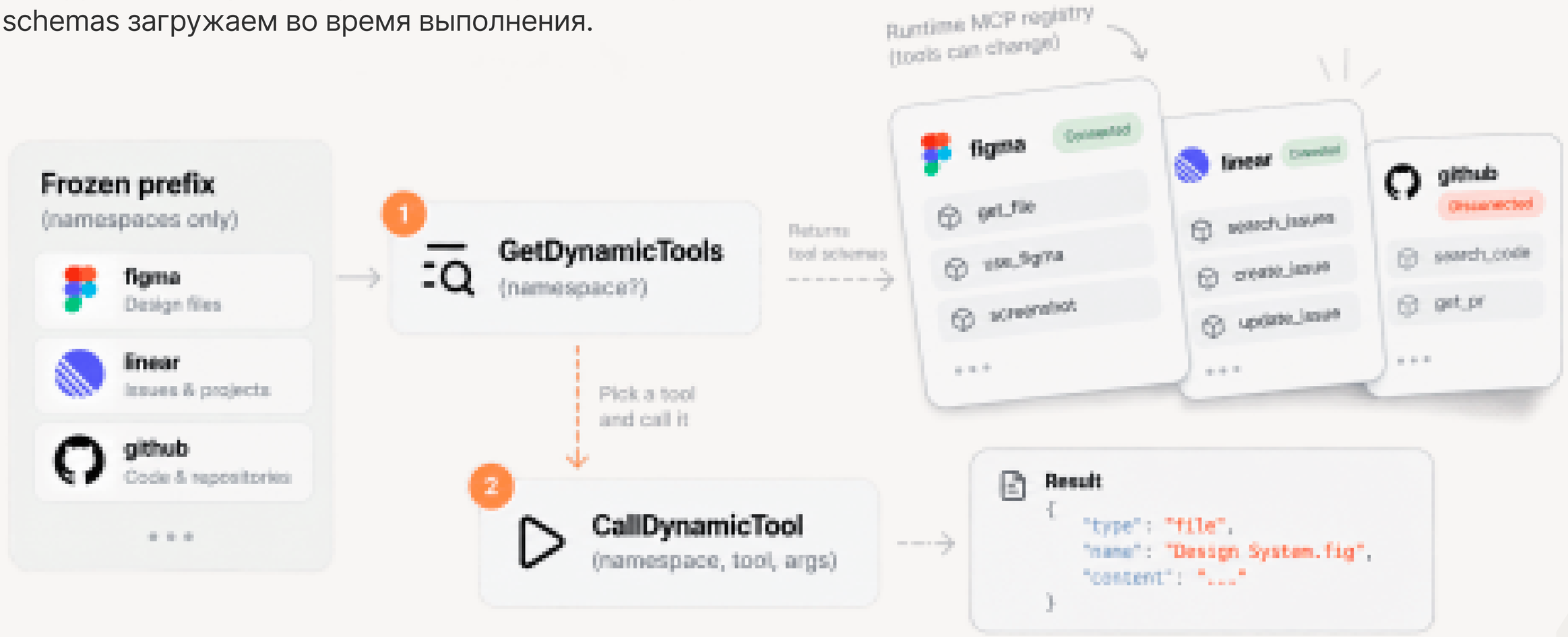
Реальный вызов

Если tool существует – исполняем вручную.

EXECUTE ✓

Два meta-tools вместо сотен schemas

Фиксируем namespaces в prefix, а доступные инструменты и schemas загружаем во время выполнения.



Не держим весь каталог tools в контексте

01 **Постоянно держим только необходимое**

Остальные tools загружаем по требованию.

02 **Привязаны к провайдеру – используем серверный ToolSearch**

Он сам находит и добавляет нужные schemas.

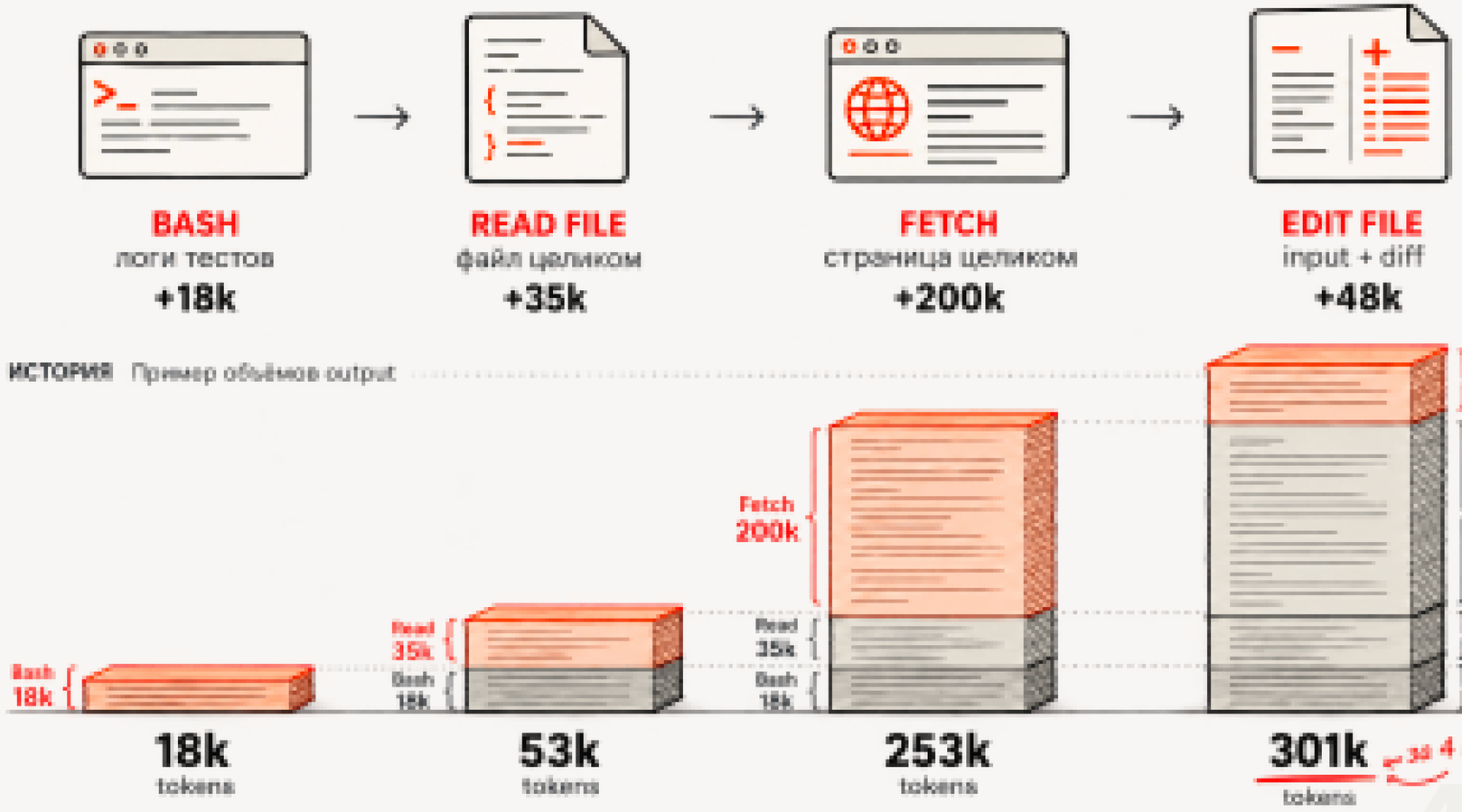
03 **Нужна независимость и работа с MCP – реализуем meta-tools**

Через них же подключаем динамические MCP

ОПТИМИЗАЦИЯ ИНСТРУМЕНТОВ

Один tool call создает тысячи лишних токенов

Инструменты это бóльшая часть истории



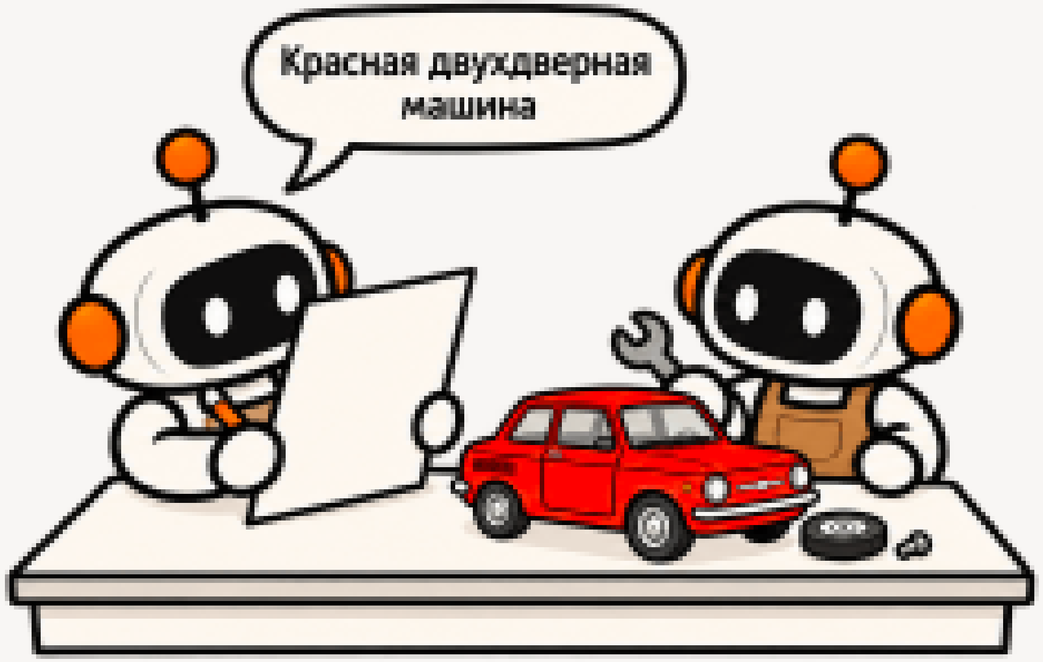
Один неосторожный вызов и его output остаётся с агентом до сжатия

Только полезный контекст

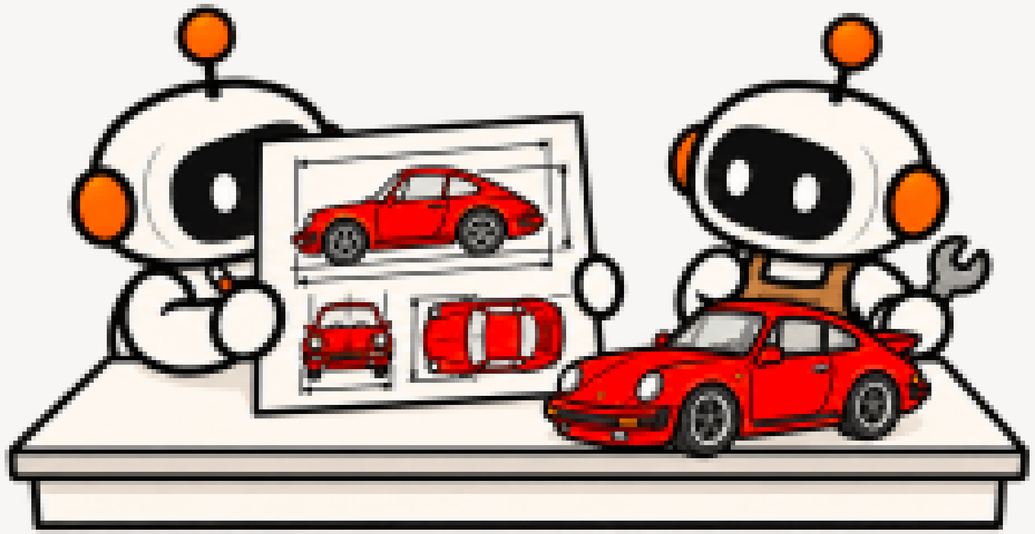
```ts
const editFile = tool({
  description: 'Replace text in a file',
  inputSchema: z.object({
    path: z.string(),
    oldText: z.string(),
    newText: z.string(),
  }),

  execute: async input => {
    const { diff, added, removed } = await applyEdit(input);

    // Полный результат — для интерфейса
    return { diff, added, removed };
  },

  // Только полезный контекст — для модели
  toModelOutput: ({ output }) => ({
    type: 'json',
    value: {
      added: output.added,
      removed: output.removed,
    },
  }),
});
```

Разделяем то что
отправляется в UI и в модель

После генерации выбираем,
что вернуть модели




```
toModelOutput: ({ output }) => {
  return {
    type: 'text',
    value: output.description,
  };
},
```



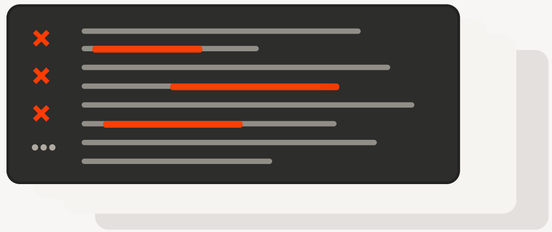
```
toModelOutput: ({ output }) => {
  return {
    type: 'content',
    value: [{
      type: 'file',
      data: { type: 'url', url: new URL(output.url) },
    }],
  };
},
```

Сжимаем Output инструмента

RAW OUTPUT

Tests
Raw output

18k tok



Filter

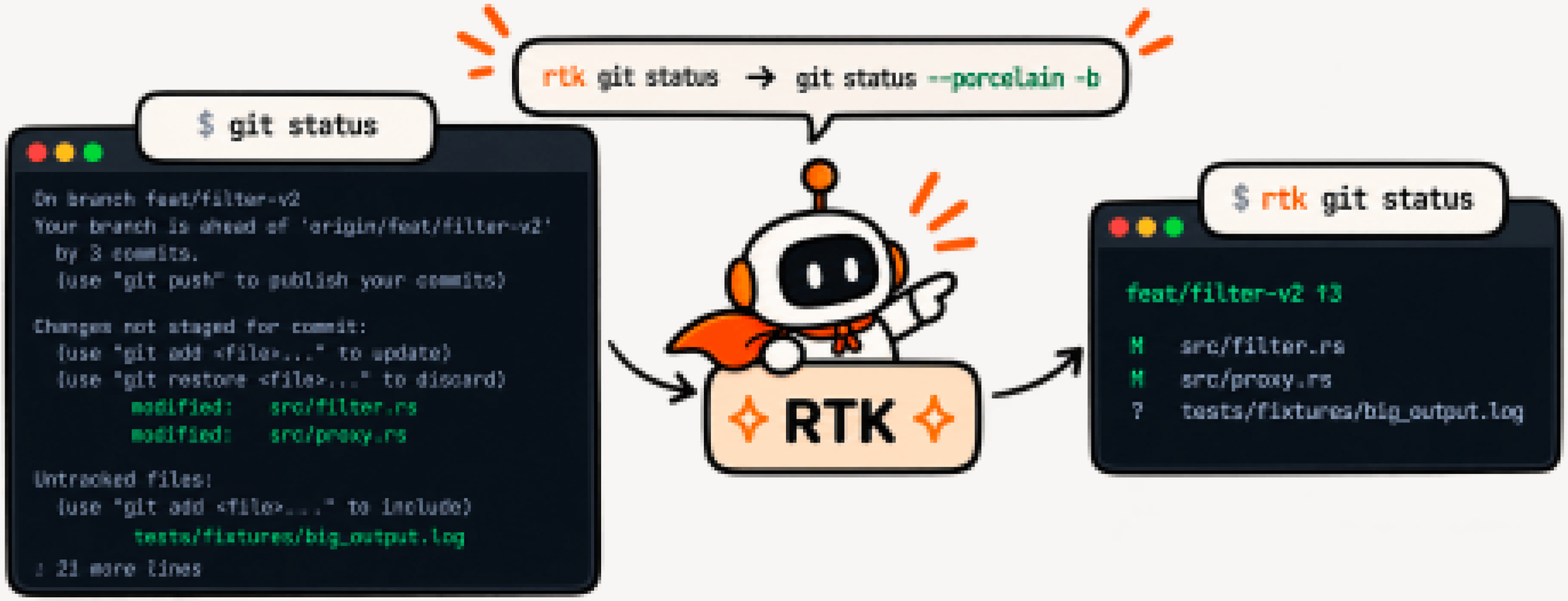
В КОНТЕКСТ

failures only

700 tok

Rust Token Killer

Модифицирует вызваемую команду и запускает его более оптимальную версию



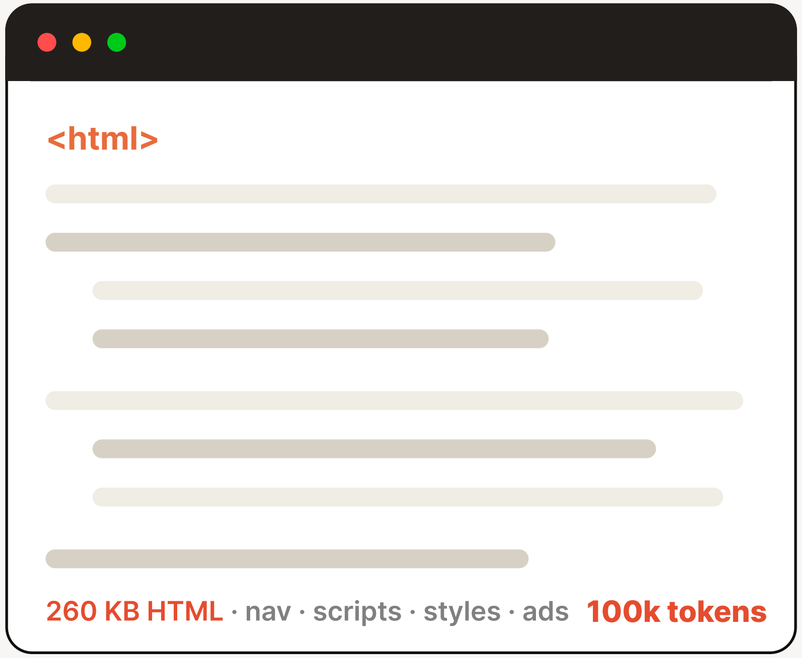
Rust Token Killer

Автоматически сжимает вывод команды убирая любые дубликаты и шум

Fetch – выкидываем boilerplate

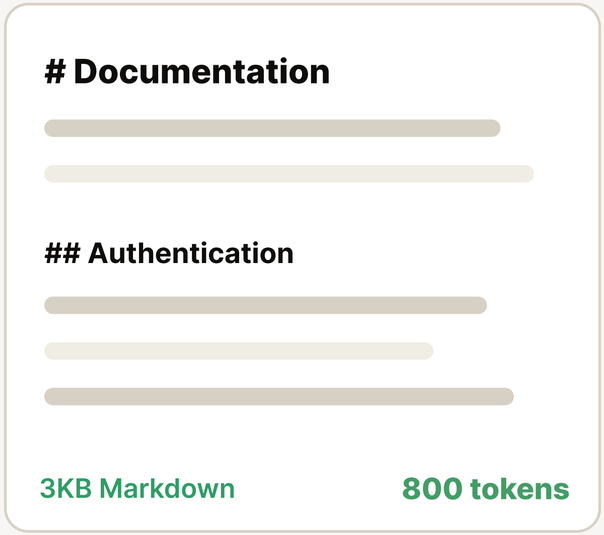
Сначала убираем разметку. Потом оставляем только то, зачем пришли

01 **HTML**
Вся страница



mdream

02 **MARKDOWN**
Только контент

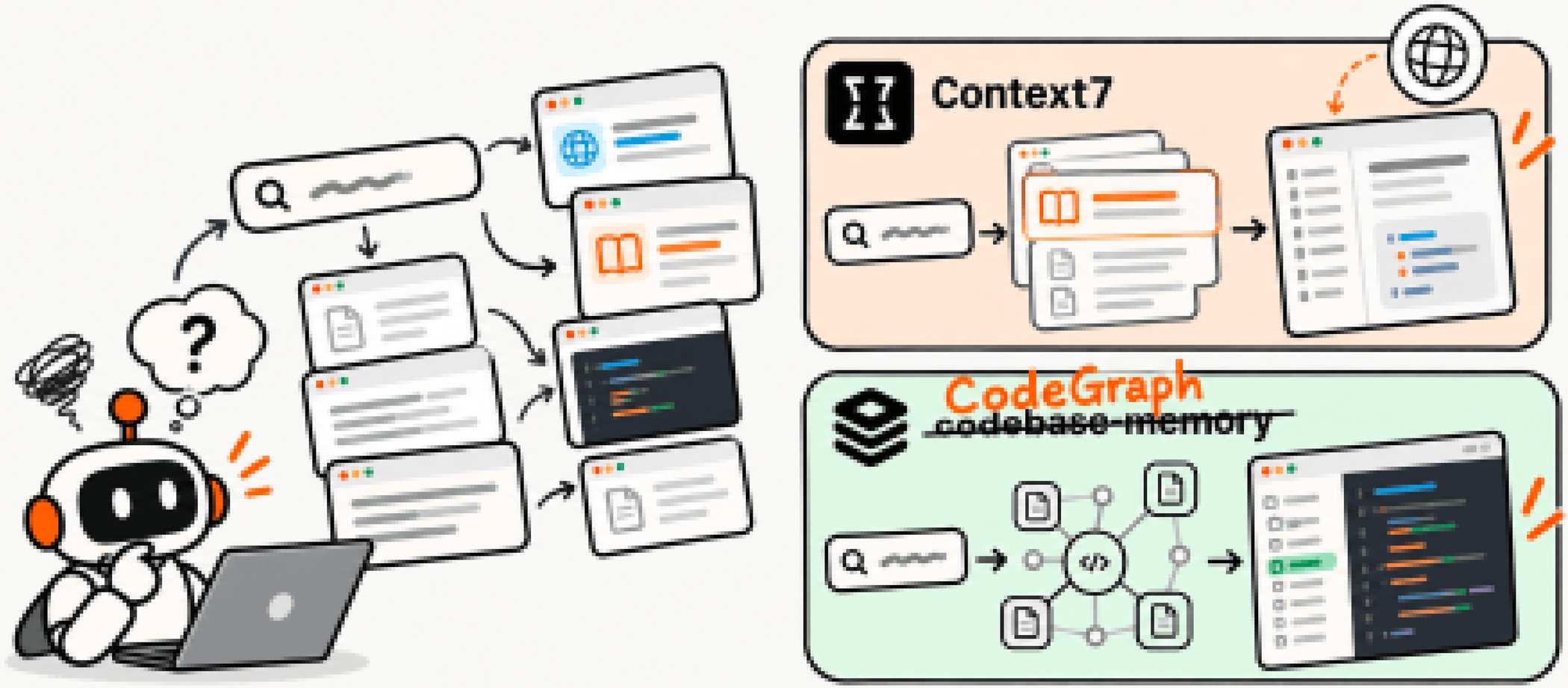


LLM Filter

03 **SUMMARY**
Только требуемая часть

Оптимизируем поиск информации

Сначала находим релевантный фрагмент и только потом отдаём его модели

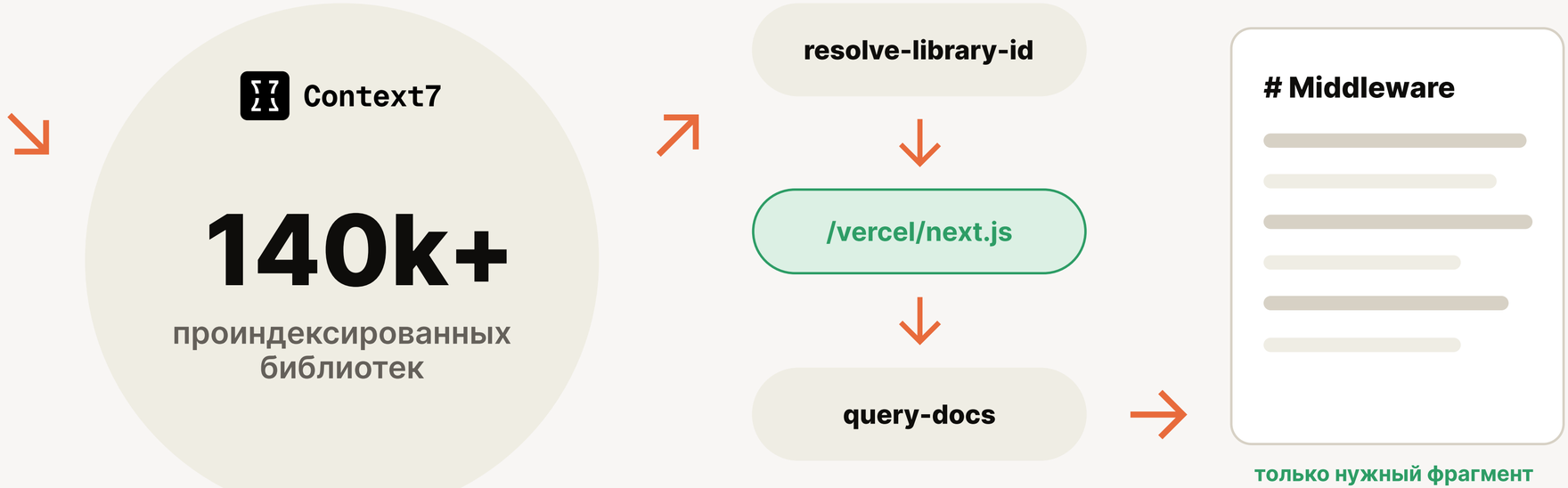


Context7: документация без веб-поиска

Вместо цепочки Search → Fetch агент сразу получает релевантный фрагмент документации

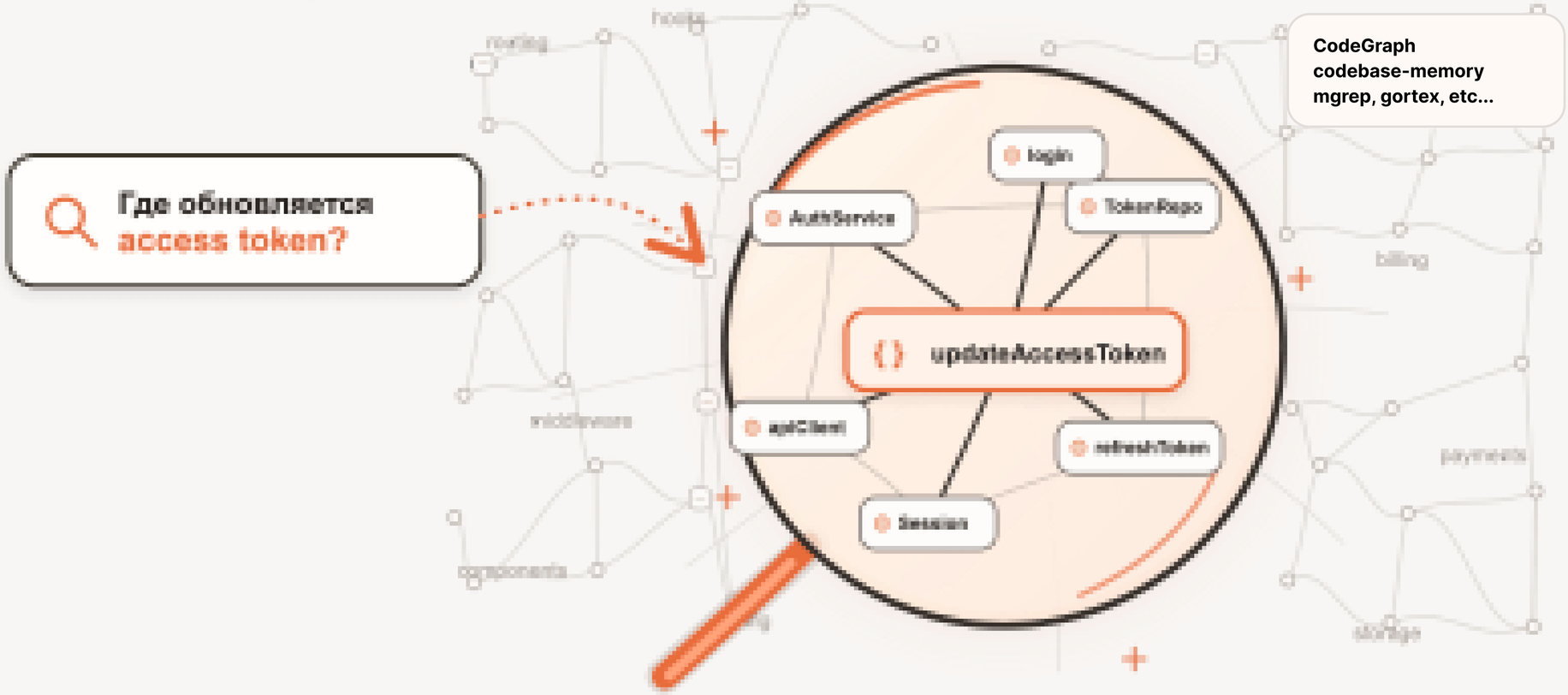
«Как работает middleware в Next.js App Router?»

запрос агента



Поиск по смыслам

Заменяем Glob / Grep на более продвинутые механики с индексом кодовой базы

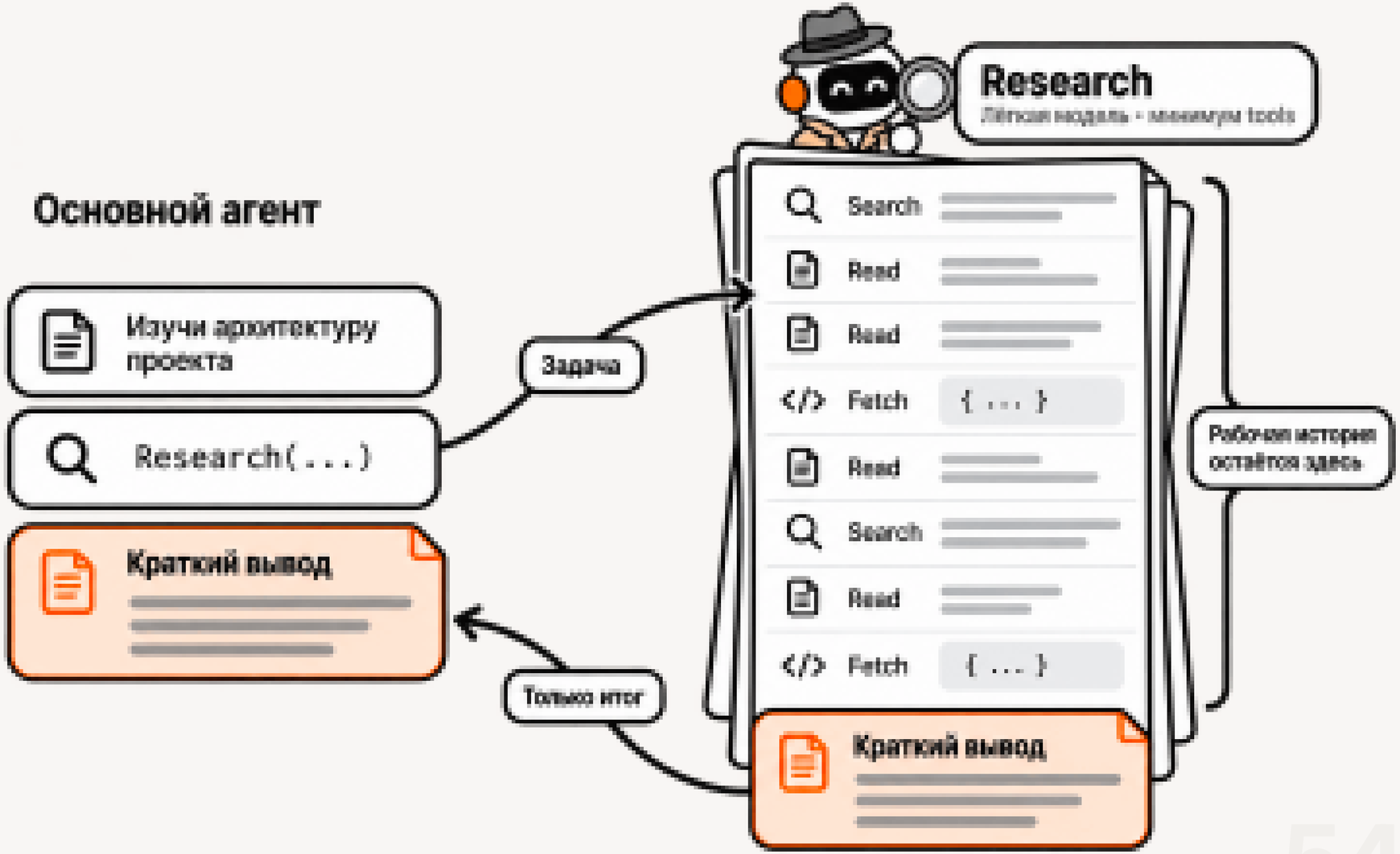


5 структурных запросов · benchmark авторов

~~412k~~ → 3,4k токенов

≈120× меньше

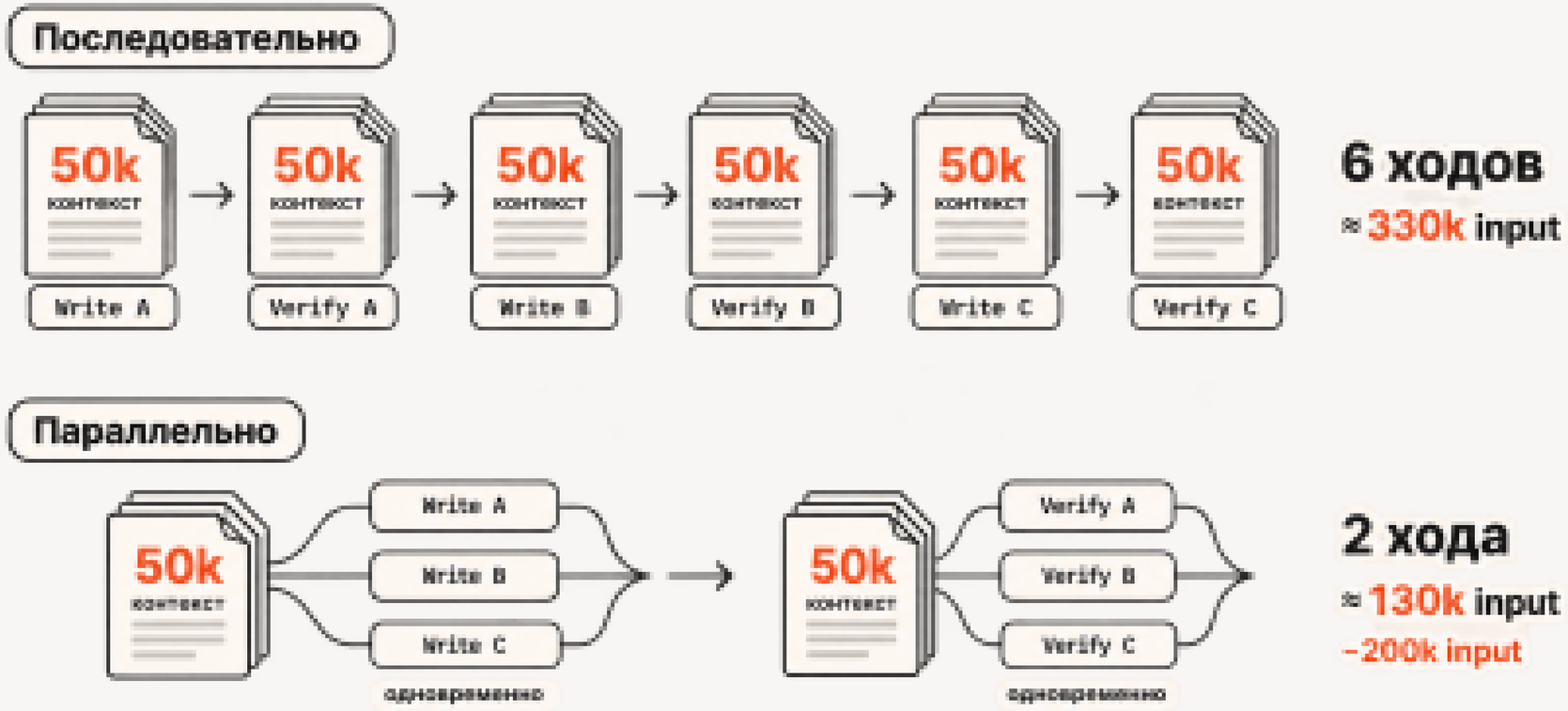
Делегируем субагентам



Вместо наполнения контекстом основного чата – делаем это в отдельном потоке

Параллельные tools экономят ходы модели

Output не уменьшается, но контекст перечитывается меньше раз



В контекст попадает только полезное

01 **Возвращайте модели минимально необходимый ей результат**

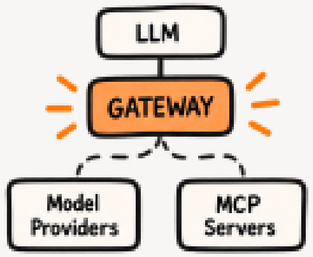
Raw-данные и UI-контент сохраняйте отдельно

02 **Исследование выносите из основного контекста**

Используйте индексы, точный поиск и субагентов

03 **Независимые tools запускайте параллельно**

Меньше ходов — меньше повторного чтения контекста



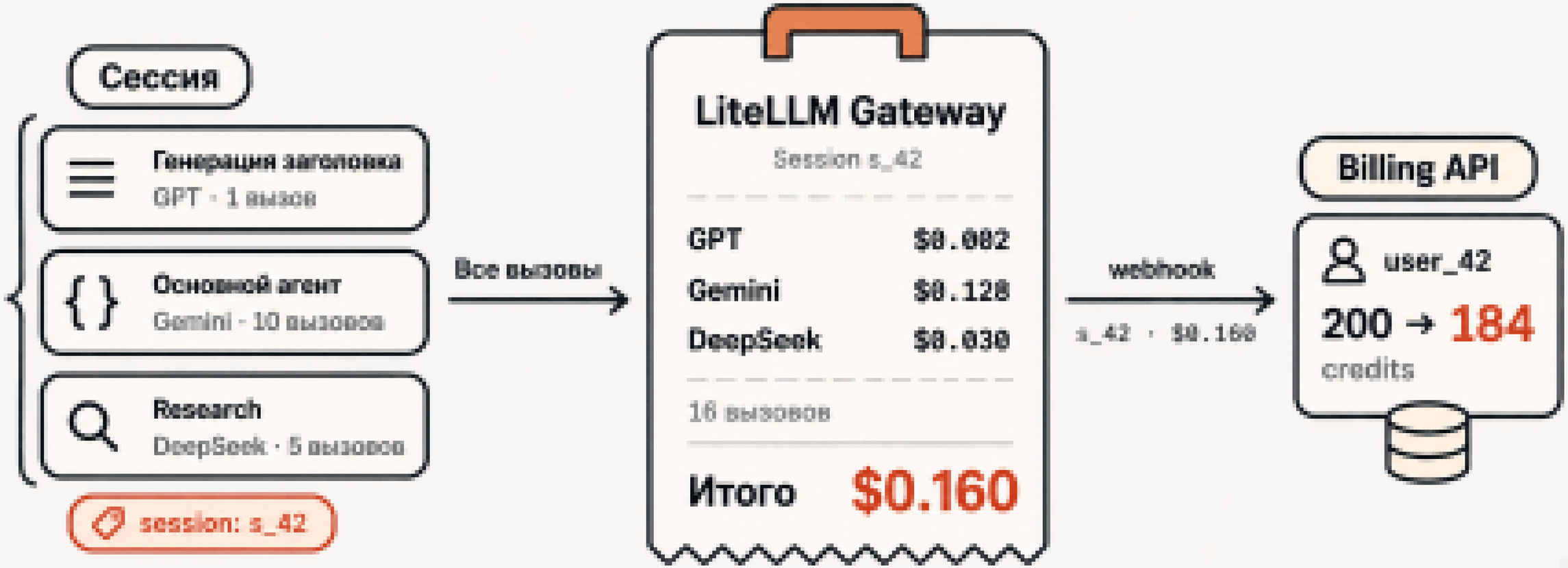
УНИФИКАЦИЯ ЗАПРОСОВ / ПОДСЧЕТ СТОИМОСТИ / ЛОГИРОВАНИЕ

Model Gateway

LiteLLM / OpenRouter

Подсчет стоимости

Gateway агрегирует расходы сессии и отправляет итог в Billing API



Один API для всех моделей

Отвязывает код от конкретной модели – модели можно менять без релизов

ЗАПРОС

POST v1/responses

model: "balanced"

input: ...

ROUTING TABLE

ALIAS	PRIMARY	FALLBACK
lite	DeepSeek 4.1-flash	GLM 5.3-flash
balanced	Gemini 3.8	gpt-6.1-sol
smart	grok-4.7	gpt-6.1-sol

ВЫБРАНО

Gemini 3.8

Код агента не изменился

Модель одна – провайдеры разные

OpenRouter динамически выбирает лучший inference endpoint

Phala 30% off	~~$0.30~~ $0.21	~~$1.20~~ $0.84	~~$0.006~~ $0.0042	1.94s	53 tps	100.00%
NovitaAI 20% off	~~$0.30~~ $0.24	~~$1.30~~ $0.96	~~$0.004~~ $0.0048	1.47s	91 tps	99.97%
Makora	$0.27	$1.15	$0.009	0.98s	114 tps	100.00%
BaiduQianfan	$0.30	$1.20	$0.006	0.93s	136 tps	99.97%
Baseten	$0.30	$1.20	$0.007	0.41s	178 tps	100.00%
AlibabaCloudInt.	$0.30	$1.20	$0.03	1.32s	106 tps	98.43%
Together	$0.30	$1.20	$0.006	0.28s	226 tps	100.00%
SiliconFlow	$0.30	$1.20	$0.006	3.88s	95 tps	99.95%
Modal	$0.30	$1.20	$0.03	0.78s	193 tps	100.00%
Baseten	$0.30	$1.20	$0.007	0.37s	160 tps	99.97%
Parasail	$0.30	$1.20	$0.006	1.25s	137 tps	100.00%
Venice	$0.30	$1.20	$0.0075	1.53s	110 tps	99.97%

СИГНАЛЫ ДЛЯ ROUTING

- PRICE
- LATENCY
- FAILURE RATE
- CACHE HIT

Серверные инструменты, доступные каждому

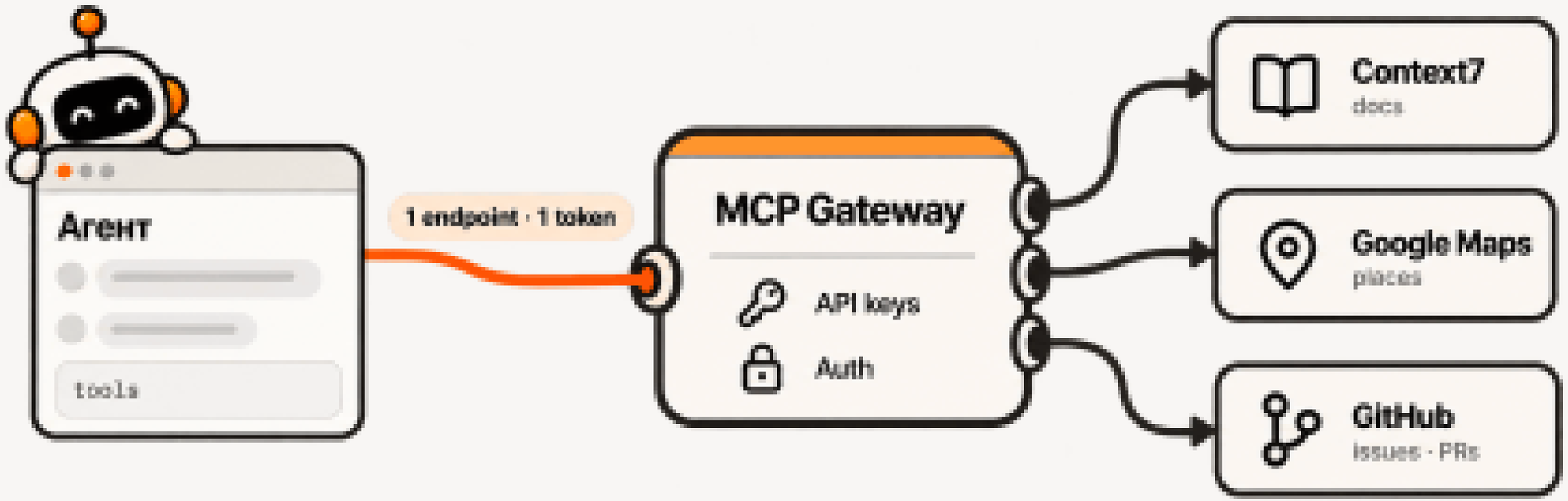
ToolSearch, WebSearch, Fetch, Subagent, и другие инструменты доступные в любой модели

Available Server Tools

Tool	Type	Description
Web Search	`openrouter:web_search`	Search the web for current information
Datetime	`openrouter:datetime`	Get the current date and time
Image Generation	`openrouter:image_generation`	Generate images from text prompts
Web Fetch	`openrouter:web_fetch`	Fetch and extract content from URLs
Apply Patch	`openrouter:apply_patch`	Propose file edits via V4A diff patches (Responses API only)
Shell	`openrouter:shell`	Run commands in a hosted, sandboxed shell (Responses and Messages APIs)
Bash	`openrouter:bash`	Anthropic-style bash tool with optional sandboxed server-side execution (Messages API only)
Fusion	`openrouter:fusion`	Run a panel of models and an analyst for multi-model analysis
Advisor	`openrouter:advisor`	Consult a stronger model for guidance mid-generation
Subagent	`openrouter:subagent`	Delegate self-contained tasks to a smaller, faster worker model
Search Models	`openrouter:experimental__search_models`	Search and filter the OpenRouter model catalog
Tool Search	`openrouter:tool_search`	Discover and load deferred tools on demand (Responses and Messages APIs)
Files	`openrouter:files`	Read, write, edit, and list text files in your workspace through the Files API

Один MCP вместо десятка

Агент знает один endpoint. Внешние сервисы подключаются за Gateway



Один Gateway — разные toolsets

Фильтруем лишние tools и собираем наборы под конкретный сценарий.

01 FILTER

Почистить конкретный MCP

Оставляем только то, что реально нужно агенту

SOURCE

Google Maps MCP

→

- ✓ search_places
- ✓ place_details
- × weather
- × routes

2 полезных tools вместо 4

02 TOOLSET

Собрать набор под сценарий

Комбинируем tools из разных MCP в один bundle

CONTEXT7
query-docs

GITHUB
search-code

PERPLEXITY
search / research

→

CUSTOM MCP BUNDLE

RESEARCH TOOLSET

docs · code · web

Только нужный набор

ВЫВОД · MODEL & MCP GATEWAY

Что забирает на себя Gateway

01 Стоимость каждого пользователя

Агрегация вызовов сессии и webhook в Billing API

02 Один API для всех моделей

Единый формат запросов и централизованные ключи

03 Модели без релизов

Aliases, primary/fallback и routing по вашим правилам

04 Лучший inference endpoint

Выбор по цене, latency, cache hit и failure rate

05 Серверные инструменты

WebSearch, Fetch, Subagent и другие server-side функции

06 MCP toolsets

Единая auth, фильтрация мусора и bundles под сценарий

ЭКОНОМИТ ВСЯ СИСТЕМА

Не один приём, а вся цепочка
от запроса до результата

01

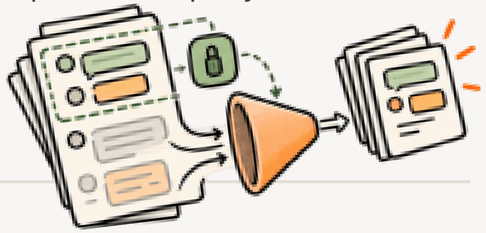
СЧИТАЕМ ВСЮ СТОИМОСТЬ

- Считаем весь Agent Loop
- Модель – под сложность

02

КОНТЕКСТ ОСТАЁТСЯ СТАБИЛЬНЫМ

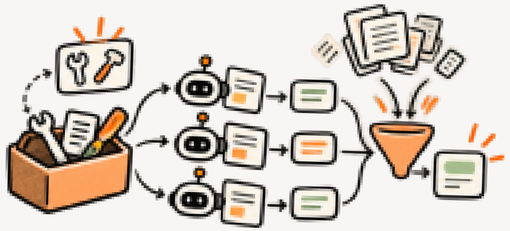
- Проверяем cache hit
- Стабильный префикс
- Сжимаем историю по порогу



03

TOOLS НЕ СОЗДАЮТ ШУМ

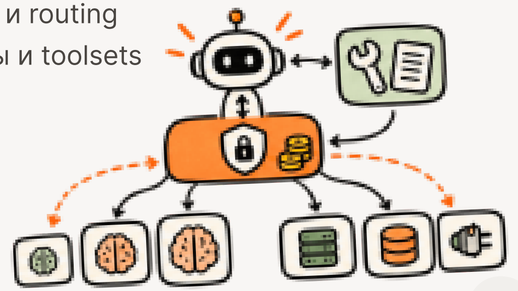
- Tools и знания – по требованию
- Сжимаем output, делеггируем субагентам



04

GATEWAY ЗАБИРАЕТ ОБВЯЗКУ

- Billing, ключи и routing
- MCP-фильтры и toolsets



Спасибо !

Вопросы ?